

claude-windows / d1586dfe-adae-4d40-a85d-ed0bb23738b8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\d1586dfe-adae-4d40-a85d-ed0bb23738b8\subagents\agent-a0a70ddb1d7197b83.jsonl`  
- SHA-256: `1667c8caa5596ccbb09c99ecc2f1eb2578a33c8646d61459a97bbc60121ebee`  
- Source modified: `2026-06-10T18:11:29+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `subagents`  
- session_id: `d1586dfe-adae-4d40-a85d-ed0bb23738b8`
```

Transcript

1. user (2026-06-10T17:59:14.460Z)

You are working in an isolated git worktree of C:\Users\avidu\Projects\muneem (a Python 3.11+ project, src layout, package src/muneem, venv at the ORIGINAL repo path
C:\Users\avidu\Projects\muneem\.venv ? use
C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe to run tools; the editable install resolves `muneem` imports from src/, which exists identically in your worktree, but to be safe run pytest with the env var PYTHONPATH pointing at YOUR worktree's src dir so your changes are the ones under test. Verify which code is being tested before trusting results: e.g. temporarily add a failing assert in your worktree to confirm it fires).

MISSION: broaden the synthetic full-pipeline regression corpus (deep-review item D1). The project parses Indian bank statement PDFs with an arithmetic verification oracle.
tests/synthpdf.py hand-rolls valid born-digital PDFs (stdlib only, real Helvetica metrics so right-aligned amount columns land where the coordinate clusterer expects).
tests/test_regression_synth.py runs them through the REAL pipeline: extract (pdfminer) ? get_parser ? parse_and_insert ? assert reconciled. Today only HDFC savings + AU savings layouts are covered, single-page, 3 perfectly-aligned rows.

Read first: tests/synthpdf.py, tests/test_regression_synth.py, src/muneem/parsers/ (savings.py, axis.py, au.py, hdfc.py, engine.py, profiles.py, validation.py, clustering.py),
src/muneem/cli.py (the ingest command + _ingest_bank_statement), src/muneem/statement_unlock.py + src/muneem/password_rules.py (the hybrid unlock), tests/test_cli.py for CLI test conventions, tests/conftest.py (autouse keychain isolation).

IMPORTANT recent contract changes you must honor (all on main):

- The savings trust oracle now ANCHORS a passing balance walk to the printed Opening/Closing Balance lines when present (savings.reconcile_oracle) ? your fixtures' first row must imply the printed opening and last row must equal the printed closing.
- parse_amount(signed=True) on balances: a "Dr" suffix = negative.
- Dates are normalized to ISO at the build boundary (parsers/dates.py).
- Axis CC persists with documents.status = "unverified" (Confidence.UNVERIFIED), and quarantines (stores nothing, "unreconciled") if any date-matched row's amount cell fails to parse.
- Parser routing requires letterhead-shaped evidence: "Axis Bank" + "Statement of Account"/"Account Statement" (title case, case-sensitive) for Axis savings; "Axis Bank" + "Credit Card Statement" for Axis CC; "au small finance bank" or "aubank.in" for AU; "hdfc bank"/"hdfcbank.com" + statement markers for HDFC.
- `txns list` tags any document status != reconciled.

DELIVERABLES (new tests in tests/test_regression_synth.py + helpers in tests/synthpdf.py):

1. ****Axis savings born-digital fixture**** routed via get_parser to AxisParser through the real pdfplumber path, reconciling + persisting (study AXIS_PROFILE in profiles.py for the column order/date format the engine expects; include "Axis Bank" + "Statement of Account" letterhead text and printed "Opening Balance"/"Closing Balance" lines consistent with the rows).
2. ****Axis credit-card fixture**** routed to AxisCreditCardParser: a 9+-column table (see axis._rows_from_cc_tables: date col 0 matching dd/mm/yyyy, particulars col 2, merchant category col 7, amount col 8 with "Dr"/"Cr" suffix) ? asserting rows persist with status "unverified".

NOTE: the CC parser uses `page.extract_tables()`, so the PDF needs ruled table lines OR you may construct this fixture at a different seam if `extract_tables` can't see a hand-rolled unruled table ? investigate what `synthpdf` can produce; if a true born-digital CC PDF through `extract_tables` is not feasible with reasonable effort, cover the CC path at the highest seam you CAN reach mechanically (document why in a comment), but try the PDF route first (`pdfplumber` `extract_tables` needs explicit lines: `synthpdf` would need to draw `rects/lines` ? PDF content streams can draw lines with simple operators like ``x y m x y l S``; this is very doable in hand-rolled PDF).

3. ****Multi-page savings statement**** (HDFC or AU style): transaction rows split across 2 pages with the header repeated ? full pipeline must still reconcile + persist the combined set.
4. ****Dropped-row-must-quarantine at the PDF level****: a statement whose printed rows are internally walk-consistent but missing a row vs the printed bookends (e.g. drop the first transaction; keep the printed Opening Balance of the full set) ? the pipeline must store NOTHING and record "unreconciled" (this reproduces the real Axis May-2025 class of failure at the PDF level and directly exercises the new bookend anchoring).
5. ****Password-protected synthetic PDF through the real CLI****: encrypt a synthetic statement with `pypdf` (AES or RC4, `pypdf` is already a dependency; e.g. `writer.encrypt("TEST1234")`) and drive ``muneem ingest <pdf> --doc-type bank_statement --db <tmp>`` via `typer CliRunner` so the REAL hybrid unlock path runs (do NOT monkeypatch `resolve_password`; inject the candidate password through a legitimate seam: read `statement_unlock.py/password_rules.py` to find it ? e.g. the `password_rules` file location override or the keychain candidate cache via the `conftest` fake keychain store fixture ``_isolate_keychain`` ? seed the cache entry the way `_remember` writes it). Assert exit 0, rows persisted, status reconciled.

RULES: offline + deterministic (no network, no real data, no real bank names beyond the routing markers, fabricated amounts), all tests in the DEFAULT suite (no new markers), `stdlib`-only PDF generation (`synthpdf` style ? `pypdf` allowed only for encryption since it's already a dep). Keep `synthpdf` helpers pure and reusable. Match the repo's comment/docstring style (explanatory, no narration).

GATE + SHIP: from the worktree run: `ruff check --fix, ruff format, mypy src`, and the full `pytest` with coverage (`.venv\Scripts\python.exe -m pytest -q --cov=muneem --cov-fail-under=80` with `PYTHONPATH` set to your worktree `src`; ALSO run plain `pytest` without coverage to be sure). All green. Then: `git checkout -b test/synthetic-corpus-breadth`, commit everything with a clear message ending in the trailer "Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>", push with `-u origin`, and create a PR with ``gh pr create`` (title "test: broaden the synthetic regression corpus (Axis savings/CC, multi-page, dropped-row, encrypted-PDF CLI)", body describing what each fixture proves, ending with the line "? Generated with [Claude Code](https://claude.com/claude-code)"). Do NOT merge the PR. If a deliverable proves genuinely infeasible after real attempts, ship the others and say exactly what blocked it in the PR body.

Your final message: the PR URL, which deliverables shipped/blocked, test counts, and any caveats. Raw facts, no pleasantries.

2. user (2026-06-10T17:59:20.359Z)

- 1 """"Dependency-free, born-digital PDF generator for regression fixtures (stdlib only).
- 2
- 3 Layer 1 of the regression-CI plan (ROADMAP § "Planned ? Regression CI") needs ****full-**
- 4 **pipeline****
- 5 fixtures ? real PDFs that go through `pdfminer/pdfplumber`, not synthetic word-lists that
- 6 bypass them.
- 7 Rather than add a dependency (`reportlab` etc.), we emit a minimal valid PDF with
- 8 ****positioned**
- 9 Helvetica text** (``Tm`/`Tj``), so it stays commercial-safe and CI-portable.
- 10 Coordinates: callers think in `pdfplumber`'s top-down ``top`` (distance from the page top);
- 11 we convert
- 12 to PDF's bottom-up baseline. Absolute ``top`` need not be exact ? parsers key on
- 13 x-positions, same-row
- 14 grouping, and row order ? but words on one row share a baseline (one ``top``) and rows are
- 15 separated.
- 16 Amount columns are ****right-aligned**** via real Helvetica metrics so their right edges
- 17 (``x1``) land in
- 18 the bands the HDFC clusterer / engine expect.

```

13     """
14
15     from __future__ import annotations
16
17     from dataclasses import dataclass
18     from pathlib import Path
19
20     # Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-
aligned amounts
21     # (digits / comma / period / minus / space); a reasonable default covers labels (left-
aligned, so
22     # their width is irrelevant to placement).
23     _W = {
24         " ": 278,
25         "!": 278,
26         "'": 355,
27         "#": 556,
28         "$": 556,
29         "%": 889,
30         "&": 667,
31         "'": 191,
32         "(": 333,
33         ")": 333,
34         "*": 389,
35         "+": 584,
36         ",": 278,
37         "-": 333,
38         ".": 278,
39         "/": 278,
40         "0": 556,
41         "1": 556,
42         "2": 556,
43         "3": 556,
44         "4": 556,
45         "5": 556,
46         "6": 556,
47         "7": 556,
48         "8": 556,
49         "9": 556,
50         ":": 278,
51         ";": 278,
52         "<": 584,
53         "=": 584,
54         ">": 584,
55         "?": 556,
56         "@": 1015,
57     }
58     _UPPER = 667 # rough average for A-Z
59     _LOWER = 528 # rough average for a-z
60
61
62     def helvetica_width(text: str, size: float) -> float:
63         """Advance width of ``text`` in Helvetica at ``size`` points."""
64         total = 0
65         for ch in text:
66             if ch in _W:
67                 total += _W[ch]
68             elif ch.isupper():
69                 total += _UPPER
70             else:
71                 total += _LOWER
72         return total / 1000.0 * size
73
74
75     @dataclass

```

```

76 class Placement:
77     """One text run: its LEFT edge ``x``, top-down ``top``, and the string."""
78
79     x: float
80     top: float
81     text: str
82     size: float = 9.0
83
84
85 def at(x: float, top: float, text: str, size: float = 9.0) -> Placement:
86     """Left-aligned run starting at ``x``."""
87     return Placement(x=x, top=top, text=text, size=size)
88
89
90 def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
91     """Right-aligned run whose right edge lands at ``x1`` (for amount columns)."""
92     return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)
93
94
95 def _esc(text: str) -> str:
96     return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")
97
98
99 def build_pdf(
100     placements: list[Placement], path: Path | str, *, width: float = 595.0, height:
float = 842.0
101 ) -> Path:
102     """Write a single-page PDF with the given positioned text. Returns the path.
103
104     Hand-rolls a minimal PDF (catalog ? pages ? page ? Helvetica font ? content stream)
with a
105     byte-accurate xref table so pdminer/pdfplumber parse it.
106     """
107     ops: list[str] = []
108     for p in placements:
109         y = height - p.top # top-down -> PDF bottom-up baseline
110         ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm
({_esc(p.text)}) Tj ET")
111     content = ("\n".join(ops)).encode("latin-1")
112
113     objects: list[bytes] = [
114         b"<< /Type /Catalog /Pages 2 0 R >>",
115         b"<< /Type /Pages /Kids [3 0 R] /Count 1 >>",
116         (
117             f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
118             f"/Resources << /Font << /F1 4 0 R >> >> /Contents 5 0 R >>"
119         ).encode("latin-1"),
120         b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
121         b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream",
122     ]
123
124     out = bytearray(b"%PDF-1.4\n")
125     offsets: list[int] = []
126     for i, obj in enumerate(objects, start=1):
127         offsets.append(len(out))
128         out += f"{i} 0 obj\n".encode() + obj + b"\nendobj\n"
129
130     xref_pos = len(out)
131     count = len(objects) + 1
132     out += f"xref\n0 {count}\n".encode()
133     out += b"0000000000 65535 f \n"
134     for off in offsets:
135         out += f"{off:010d} 00000 n \n".encode()
136     out += (

```

```

137         b"trailer\n<< /Size " + str(count).encode() + b" /Root 1 0 R >>\n"
138         b"startxref\n" + str(xref_pos).encode() + b"\n%%EOF"
139     )
140
141     out_path = Path(path)
142     out_path.write_bytes(out)
143     return out_path
144
145
146     __all__ = ["Placement", "at", "right", "helvetica_width", "build_pdf"]
147

```

3. user (2026-06-10T17:59:21.119Z)

```

1     """Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.
2
3     Generates *real* PDFs (no committed statements, no secrets) and runs the production
4     pipeline ?
5     ``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled**
6     parse. This
7     covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the
8     synthetic
9     word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP §
10    "Planned ?
11    Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud
12    CI.
13    """
14
15    from pathlib import Path
16
17    from muneem import db as dbmod
18    from muneem.extract import extract
19    from muneem.parsers import get_parser
20    from synthpdf import at, build_pdf, right
21
22    def _hdfc_pdf(path: Path) -> Path:
23        """HDFC layout: leading date x0?52, narration x0?111, three right-aligned amount
24        columns whose
25        right edges land in the clusterer's bands (withdrawal <400, deposit <490, balance
26        ?490), plus a
27        SUMMARY block for the bookend. Three rows reconcile (opening 10,000 ? closing
28        10,000)."""
29        p = [
30            at(50, 60, "HDFC BANK"),
31            at(50, 72, "Savings Account Details"),
32            at(50, 84, "Account Number : 50100012345678"),
33            # header ? must carry "Narration" + a "Closing" token so _table_body locates the
34            table
35            at(72, 110, "Date"),
36            at(175, 110, "Narration"),
37            at(300, 110, "Withdrawals"),
38            at(390, 110, "Deposits"),
39            at(480, 110, "Closing"),
40            at(515, 110, "Balance"),
41        ]
42        rows = [
43            ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
44            ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
45            ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
46        ]
47        top = 130
48        for d, narr, wd, dep, bal in rows:
49            p += [
50                at(52, top, d),

```

```

43         at(111, top, narr),
44         right(358, top, wd),
45         right(443, top, dep),
46         right(561, top, bal),
47     ]
48     top += 15
49     p += [
50         at(50, 200, "SUMMARY"),
51         at(50, 212, "Opening Balance Debit Amount Credit Amount Closing Balance"),
52         at(50, 224, "10,000.00 2,000.00 2,000.00 10,000.00"),
53         at(50, 236, "Debit Count Credit Count"),
54         at(50, 248, "2 1"),
55     ]
56     return build_pdf(p, path)
57
58
59     def _au_savings_pdf(path: Path) -> Path:
60         """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
61         date | narration | debit | credit | balance table in well-separated columns ?
62         clustering ? engine mapping ? per-row reconcile. Three rows that walk (opening
63         50,000)."""
64         p = [
65             at(50, 60, "AU Small Finance Bank"),
66             at(50, 72, "Account Number : 1234567890123"),
67             # header (clustering's date+money filter drops it; here only for realism / text
68             markers)
69             at(52, 100, "Date"),
70             at(110, 100, "Transaction"),
71             at(310, 100, "Withdrawal"),
72             at(395, 100, "Deposit"),
73             at(485, 100, "Balance"),
74         ]
75         rows = [
76             ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
77             ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
78             ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
79         ]
80         top = 124
81         for d, narr, debit, credit, bal in rows:
82             line = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
83             if debit:
84                 line.append(right(340, top, debit))
85             if credit:
86                 line.append(right(420, top, credit))
87             p += line
88             top += 15
89         return build_pdf(p, path)
90
91     def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
92         """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back
93         status."""
94         ex = extract(pdf)
95         parser = get_parser(ex.text)
96         assert parser is not None, "no parser routed this synthetic statement"
97         with dbmod.connect(":memory:") as conn:
98             dbmod.init_db(conn)
99             doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256,
100             status="parsed")
101             n = parser.parse_and_insert(pdf, conn, doc_id)
102             status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
103         return type(parser).__name__, n, status

```

```

103     def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
104         name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
105         assert name == "HDFCParser"
106         assert n == 3, f"expected 3 transactions, got {n}"
107         assert status == "reconciled", f"expected reconciled, got {status}"
108
109
110     def test_savings_layout_full_pipeline_reconciles(tmp_path):
111         name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
112         assert name == "AUParser"
113         assert n == 3, f"expected 3 transactions, got {n}"
114         assert status == "reconciled", f"expected reconciled, got {status}"
115

```

4. user (2026-06-10T17:59:26.006Z)

```

1     """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3     Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4     and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6     1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7         coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8     2. The generic **engine** maps each candidate's columns onto the canonical concepts.
9     3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
10         vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12     A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13     on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
14     cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
15     can't yet read returns ``verified=False`` and the caller persists nothing ? never
misabeled rows.
16
17     No statement content is logged; only counts/booleans travel out.
18     """
19
20     from __future__ import annotations
21
22     import re
23     from dataclasses import dataclass, field
24     from pathlib import Path
25
26     import pdfplumber
27
28     from .clustering import cluster_to_matrix
29     from .concepts import ConceptRow, StatementConcepts
30     from .dates import normalize_statement_date
31     from .engine import Profile, map_table
32     from .validation import (
33         BALANCE_TOLERANCE,
34         parse_amount,
35         reconcile_running_balance,
36         reconcile_total,
37     )
38
39     _MONEY_TOKEN = re.compile(r"^\-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
40     # Capture an optional trailing Cr/Dr marker: "Opening Balance: 5,000.00 Dr" is NEGATIVE

```

```

41     # (overdrawn) and must anchor the oracle with its true sign.
42     _OPENING_RE = re.compile(
43         r"opening\s+balance[^\d\-\-]*(-?\[\d,\]+\.d{2})(\s*(?:cr|dr))?", re.IGNORECASE
44     )
45     _CLOSING_RE = re.compile(
46         r"closing\s+balance[^\d\-\-]*(-?\[\d,\]+\.d{2})(\s*(?:cr|dr))?", re.IGNORECASE
47     )
48
49
50     def _summary_balance(m: re.Match | None) -> float | None:
51         return parse_amount(f"{m.group(1)}{m.group(2) or ''}", signed=True) if m else None
52
53
54     @dataclass
55     class SavingsParse:
56         """Outcome of a savings parse: concept rows + whether reconciliation endorsed
57         them."""
58         account_number: str
59         rows: list[ConceptRow] = field(default_factory=list)
60         verified: bool = False
61
62
63     def parse_statement_summary(text: str) -> StatementConcepts:
64         """Read opening/closing balance from statement text (for the total-reconcile
65         oracle).
66         Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
67         walk.
68         """
69         return StatementConcepts(
70             opening_balance=_summary_balance(_OPENING_RE.search(text)),
71             closing_balance=_summary_balance(_CLOSING_RE.search(text)),
72         )
73
74     def txn_line_predicate(profile: Profile):
75         """A line is transaction-like if it carries one of the profile's dates *and* a money
76         token.
77         Used to restrict coordinate clustering to the transaction band (drops headers /
78         summaries /
79         address blocks that would otherwise wreck column inference).
80         """
81         patterns = profile.date_patterns
82
83         def keep(texts: list[str]) -> bool:
84             has_date = any(any(p.match(t) for p in patterns) for t in texts)
85             return has_date and any(_MONEY_TOKEN.match(t) for t in texts)
86
87         return keep
88
89     def reconcile_oracle(opening: float | None, closing: float | None, tol: float =
90     BALANCE_TOLERANCE):
91         """Trust a parse only when the arithmetic AND every available printed bookend agree.
92         The per-row walk proves *internal* consistency, but it cannot see rows missing at
93         the
94         statement's **edges** ? dropping the first or last transaction leaves a perfectly
95         consistent chain. So a passing walk is additionally anchored to the printed
96         opening/closing balance whenever the summary supplied them: the first row must imply
97         the printed opening (``balance ? credit + debit``), the last row must equal the
98         printed
99         closing. Without a usable walk, both bookends + the total identity carry the

```



```

verdict.
98
99     This is the **trust** oracle for a whole-statement row set. Per-table *selection*
during
100     multi-page mapping uses :func:`_selection_oracle` instead ? a page fragment can't
span
101     the bookends.
102     """
103
104     def check(rows: list[ConceptRow]) -> bool:
105         if not rows:
106             return False
107         if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
108             first, last = rows[0], rows[-1]
109             if opening is not None and first.balance is not None:
110                 implied = first.balance - (first.credit or 0.0) + (first.debit or 0.0)
111                 if abs(implied - opening) > tol:
112                     return False
113             if closing is not None and last.balance is not None:
114                 if abs(last.balance - closing) > tol:
115                     return False
116             return True
117         if opening is not None and closing is not None:
118             return reconcile_total(rows, opening, closing)
119         return False
120
121     return check
122
123
124     def _selection_oracle(opening: float | None, closing: float | None):
125         """Endorse a candidate column-mapping for ONE table (possibly a page fragment).
126
127         A fragment's walk proves the mapping is right even though the fragment can't span
128         the
129         printed bookends, so no edge anchoring here ? the combined row set is re-judged by
130         the
131         strict :func:`reconcile_oracle` before anything is trusted.
132         """
133
134         def check(rows: list[ConceptRow]) -> bool:
135             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
136                 return True
137             if rows and opening is not None and closing is not None:
138                 return reconcile_total(rows, opening, closing)
139             return False
140
141         return check
142
143     def _reconstruct_by_balance_delta(
144         matrix: list[list[str]], profile: Profile, opening: float | None, tol: float =
BALANCE_TOLERANCE
145     ) -> list[ConceptRow]:
146         """Recover debit/credit from the running balance when columns won't split cleanly.
147
148         Needs only three things per row ? a date, the transaction amount, and the running
149         balance ? so
150         it tolerates messy column inference (a sliver column, an interleaved value-date
151         column, or a
152         single merged withdrawal/deposit column). Direction is the sign of the balance
153         change; the
154         caller's reconciliation then **cross-checks** that the amount equals |?balance| on
155         every row, so
156         a misread or a dropped row still fails (it is *not* circular). Needs the printed
157         opening balance

```

```

152         to fix the first row's direction; returns ``[]`` if it can't form a clean series.
153         ""
154         patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
155         rows = [(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
156         if not rows or opening is None:
157             return []
158         ncols = max(len(r) for r in rows)
159
160         def is_date(s: str) -> bool:
161             return any(p.match(s) for p in patterns)
162
163         def money(s: str) -> float | None:
164             return parse_amount(s) if _MONEY_TOKEN.match(s) else None
165
166         def balance_money(s: str) -> float | None:
167             # Balance cells honour a Cr/Dr suffix as a SIGN (Dr = overdrawn = negative).
168             return parse_amount(s, signed=True) if _MONEY_TOKEN.match(s) else None
169
170         # date column = the leftmost of the (first few) columns with the most date cells
171         date_counts: dict[int, int] = {}
172         for r in rows:
173             for c in range(min(ncols, 3)):
174                 if c < len(r) and is_date(r[c]):
175                     date_counts[c] = date_counts.get(c, 0) + 1
176         if not date_counts:
177             return []
178         top = max(date_counts.values())
179         date_col = min(c for c, n in date_counts.items() if n == top)
180
181         txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
182         if len(txn_rows) < 2:
183             return []
184
185         # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
186         # column still counts, but an interleaved value-date column (mostly dates) does not.
The
187         # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
188         money_cols = []
189         for c in range(ncols):
190             m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
191             d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
192             if m >= 1 and m > d:
193                 money_cols.append(c)
194         if len(money_cols) < 2:
195             return []
196         balance_col = max(money_cols)
197         flow_cols = [c for c in money_cols if c != balance_col]
198
199         text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
200         narr_col = (
201             max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
202             if text_cols
203             else None
204         )
205
206         out: list[ConceptRow] = []
207         prev = opening
208         for r in txn_rows:
209             bal = balance_money(r[balance_col]) if balance_col < len(r) else None
210             if bal is None:
211                 return [] # a running balance on every row is required for the delta
212             amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)

```

```

213         debit = credit = None
214         if amt is not None:
215             if bal < prev - tol:
216                 debit = amt
217             elif bal > prev + tol:
218                 credit = amt
219             else:
220                 debit = amt # zero net with an amount present ? reconciliation will
flag it
221         out.append(
222             ConceptRow(
223                 txn_date=normalize_statement_date(r[date_col]) or "",
224                 narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
225                 debit=debit,
226                 credit=credit,
227                 balance=bal,
228             )
229         )
230         prev = bal
231     return out
232
233
234     def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
235         """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.
236
237         Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
238         and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
239         reconciles, so the caller can refuse to persist.
240         """
241         with pdfplumber.open(pdf_path, password=password) as pdf:
242             words_by_page = [
243                 page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
244             ]
245             tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
246             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
247
248             account_number = profile.first_account(head) or "UNKNOWN"
249             summary = parse_statement_summary(head)
250             trust = reconcile_oracle(summary.opening_balance, summary.closing_balance)
251             select = _selection_oracle(summary.opening_balance, summary.closing_balance)
252
253             confirmed: list[ConceptRow] = []
254             for table in tables:
255                 res = map_table(table, profile, oracle=select) # fragment-level mapping
selection
256                 if res and res.verified:
257                     confirmed.extend(res.rows)
258             if confirmed and trust(confirmed): # the combined set must pass the bookend-
anchored oracle
259                 return SavingsParse(account_number, confirmed, True)
260
261             # The clustered matrix and the delta reconstruction span the whole statement, so the
strict
262             # bookend-anchored oracle judges them directly.
263             matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
264             clustered = map_table(matrix, profile, oracle=trust)
265             if clustered and clustered.verified:
266                 return SavingsParse(account_number, clustered.rows, True)

```

```

267
268         # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
269         #     debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
270         #     direction from the running balance and let the oracle cross-check amount ==
|?balance|.
271         delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
272         if delta and trust(delta):
273             return SavingsParse(account_number, delta, True)
274
275         fallback = confirmed or (clustered.rows if clustered else [])
276         return SavingsParse(account_number, fallback, False)
277
278
279     __all__ = [
280         "SavingsParse",
281         "parse_savings",
282         "parse_statement_summary",
283         "txn_line_predicate",
284         "reconcile_oracle",
285     ]
286

```

5. user (2026-06-10T17:59:26.665Z)

```

1     """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
S6).
2
3     A profile carries the priors the generic engine needs to disambiguate one issuer: extra
4     concept?label aliases, the accepted date shape, a positional fallback when there's no
header,
5     table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
an issuer is
6     ideally *adding a profile*, not writing a parser.
7
8     `HDFC_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke
coordinate-
9     clustering ? the documented exception where `extract_tables()` fails on a semi-ruled
table (see the
10     `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their
engine-driven
11     parsers (coordinate clustering ? generic mapping ? total reconcile).
12     """
13
14     from __future__ import annotations
15
16     import re
17
18     from .engine import Profile
19
20     AXIS_PROFILE = Profile(
21         name="axis",
22         aliases={
23             # Real Axis savings header: Tran/Txn Date | Transaction | Withdrawals | Deposits
| Balance |
24             # Other Information. "Transaction" is the narration column (not the date).
25             "narration": ("transaction", "particulars"),
26             "debit": ("withdrawals", "withdrawal", "debit"),
27             "credit": ("deposits", "deposit", "credit"),
28             "balance": ("balance",),
29         },
30         date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),), # dd-mm-yyyy or
dd/mm/yyyy
31         # Layout / column-count varies between vintages, so there is NO reliable

```

```

column_order ? the
32         # generic content search + total reconcile do the work. Broadened account pattern:
the old
33         # `Account No:` regex missed real statements.
34         account_patterns=(
35             re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?s*(\d{6,})", re.IGNORECASE),
36             re.compile(r"A/c\s*No\.\.?s*[:\s]*(\d{6,})", re.IGNORECASE),
37         ),
38         quirks=frozenset({"layout-varies", "other-information-column", "balance-optional"}),
39     )
40
41     AUBANK_PROFILE = Profile(
42         name="aubank",
43         aliases={
44             "narration": (
45                 "transaction",
46                 "particulars",
47                 "description",
48                 "narration",
49                 "transaction details",
50             ),
51             "debit": ("withdrawal", "withdrawals", "debit", "withdrawal amount", "dr"),
52             "credit": ("deposit", "deposits", "credit", "deposit amount", "cr"),
53             "balance": ("balance", "closing balance", "running balance"),
54         },
55         # AU vintages use dd-mm-yyyy / dd/mm/yyyy or dd-Mon-yyyy; verified on the real
statements.
56         date_patterns=(
57             re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),
58             re.compile(r"^\d{2}-[A-Za-z]{3}-\d{4}$"),
59         ),
60         account_patterns=(
61             re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?s*(\d{6,})", re.IGNORECASE),
62             re.compile(r"A/c\s*No\.\.?s*[:\s]*(\d{6,})", re.IGNORECASE),
63         ),
64         quirks=frozenset({"engine-driven"}),
65     )
66
67     HDFC_PROFILE = Profile(
68         name="hdfc",
69         aliases={
70             "debit": ("withdrawal", "withdrawals"),
71             "credit": ("deposit", "deposits"),
72             "balance": ("closing balance",),
73         },
74         date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
75         table_anchors=("Narration", "Closing Balance"),
76         account_patterns=(re.compile(r"Account\s*Number\s*[:.]\s*(\d{6,})",
re.IGNORECASE),),
77         # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
78         # by word-position clustering rather than through the generic cell-matrix engine.
79         quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
80     )
81
82     __all__ = ["AXIS_PROFILE", "AUBANK_PROFILE", "HDFC_PROFILE"]
83

```

6. user (2026-06-10T17:59:33.223Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-

```

```

gated** so a
5     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
Axis specifics
6     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
mapping.
7
8     Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
reconcile
9     (parse/persist split + count only). No statement content is logged here.
10    ""
11
12    from __future__ import annotations
13
14    import re
15    import sqlite3
16    from dataclasses import dataclass
17    from pathlib import Path
18
19    import pdfplumber
20
21    from muneem import __version__
22    from muneem import db as dbmod
23    from muneem.errors import ParseError
24    from muneem.redaction import redact_filename
25
26    from . import register_parser
27    from .base import StatementParser
28    from .dates import normalize_statement_date
29    from .profiles import AXIS_PROFILE
30    from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
31    from .validation import Confidence, parse_amount
32
33    _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
34    _CARD_RES = [re.compile(r"(\d{4}XXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
35
36
37    def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
38        for rx in patterns:
39            m = rx.search(text)
40            if m:
41                return m.group(1).strip()
42        return None
43
44
45    # --- Savings / transaction account (concept core)
-----
46
47
48    # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
49    AxisSavings = SavingsParse
50    parse_axis_summary = parse_statement_summary
51    _is_txn_line = txn_line_predicate(AXIS_PROFILE)
52
53
54    def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
55        """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
56        return parse_savings(pdf_path, AXIS_PROFILE, password)
57
58
59    @register_parser
60    class AxisParser(StatementParser):
61        def can_handle(self, text: str, sender: str | None = None, subject: str | None =

```

```

None) -> bool:
62         if sender and "axisbank.com" in sender.lower():
63             return True
64         t = text or ""
65         return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
66
67     def parse_and_insert(
68         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
69     ) -> int:
70         """Persist a savings parse **only if it reconciles**; otherwise store nothing.
71
72         Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
73         parser can't yet read surfaces loudly instead of saving mislabeled rows.
74         """
75         try:
76             parsed = parse_axis_savings(pdf_path, password)
77         except ParseError:
78             raise
79         except Exception as exc:
80             # Filename only ? never statement text; original cause kept via `from`.
81             name = redact_filename(pdf_path)
82             raise ParseError(f"failed to parse Axis statement {name}") from exc
83
84         if not parsed.verified:
85             dbmod.record_parse(
86                 conn,
87                 doc_id,
88                 confidence=Confidence.UNRECONCILED.value,
89                 parser="AxisParser",
90                 parser_version=__version__,
91             )
92         return 0
93
94         for r in parsed.rows:
95             dbmod.insert_axis_transaction(
96                 conn,
97                 document_id=doc_id,
98                 account_number=parsed.account_number,
99                 txn_date=r.txn_date,
100                 value_date=r.value_date or "",
101                 particulars=r.narration,
102                 debit=r.debit,
103                 credit=r.credit,
104                 balance=r.balance,
105             )
106             dbmod.record_parse(
107                 conn,
108                 doc_id,
109                 confidence=Confidence.RECONCILED.value,
110                 parser="AxisParser",
111                 parser_version=__version__,
112             )
113         return len(parsed.rows)
114
115
116     # --- Credit card (no running balance; count only)
-----
117
118
119     @dataclass
120     class AxisCCTxn:
121         txn_date: str

```

```

122     particulars: str
123     merchant_category: str
124     amount: float
125     type: str # "Cr" or "Dr"
126
127
128     def _rows_from_cc_tables(tables: list) -> tuple[list[AxisCCTxn], int]:
129         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB.
130
131         Returns ``(rows, bad_amounts)`` ? ``bad_amounts`` counts transactional rows (date-
132         matched)
133         whose amount cell did not parse. With no balance walk to catch a misread, a silent
134         ``0.0``
135         coercion would persist a wrong amount as if real; the caller must treat any bad
136         amount as
137         a suspect parse.
138         """
139         rows: list[AxisCCTxn] = []
140         bad_amounts = 0
141         for table in tables or []:
142             for row in table or []:
143                 if not row or len(row) < 9:
144                     continue
145                 date_str = str(row[0]).strip() if row[0] else ""
146                 if not _CC_DATE_RE.match(date_str):
147                     continue
148                 amount_str = str(row[8]).strip() if row[8] else ""
149                 is_cr = "Cr" in amount_str
150                 amount = parse_amount(re.sub(r"^\d.", "", amount_str))
151                 if amount is None:
152                     bad_amounts += 1
153                     continue
154                 rows.append(
155                     AxisCCTxn(
156                         txn_date=date_str,
157                         particulars=str(row[2]).strip() if row[2] else "",
158                         merchant_category=str(row[7]).strip() if row[7] else "",
159                         amount=amount,
160                         type="Cr" if is_cr else "Dr",
161                     )
162                 )
163             return rows, bad_amounts
164
165
166     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
167     list[AxisCCTxn], int]:
168         """Open an Axis credit-card statement; return (card_number, transactions,
169         bad_amounts)."""
170         card_number = "UNKNOWN"
171         rows: list[AxisCCTxn] = []
172         bad_amounts = 0
173         with pdfplumber.open(pdf_path, password=password) as pdf:
174             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
175             len(pdf.pages))))
176             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
177             for page in pdf.pages:
178                 page_rows, page_bad = _rows_from_cc_tables(page.extract_tables())
179                 rows.extend(page_rows)
180                 bad_amounts += page_bad
181             return card_number, rows, bad_amounts
182
183
184     @register_parser
185     class AxisCreditCardParser(StatementParser):
186         def can_handle(self, text: str, sender: str | None = None, subject: str | None =

```



```

None) -> bool:
181         return "Axis Bank" in text and "Credit Card Statement" in text
182
183     def parse_and_insert(
184         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
185     ) -> int:
186         """Persist CC rows with an explicit ``unverified`` verdict ? never as silently
fine.
187
188         A credit-card bill has no running balance, so no arithmetic oracle can endorse
the
189         parse (Confidence.UNVERIFIED, surfaced by every listing). What CAN be checked is
190         checked: a transactional row whose amount cell does not parse means the layout
read
191         is suspect ? the parse is quarantined (store nothing, ``unreconciled``) rather
than
192         persisting a wrong amount the math could never catch.
193         """
194         try:
195             card_number, rows, bad_amounts = parse_axis_cc(pdf_path, password)
196         except ParseError:
197             raise
198         except Exception as exc:
199             raise ParseError(
200                 f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
201             ) from exc
202         if bad_amounts > 0:
203             dbmod.record_parse(
204                 conn,
205                 doc_id,
206                 confidence=Confidence.UNRECONCILED.value,
207                 parser="AxisCreditCardParser",
208                 parser_version=__version__,
209             )
210         return 0
211         for t in rows:
212             dbmod.insert_axis_cc_transaction(
213                 conn,
214                 document_id=doc_id,
215                 account_number=card_number,
216                 txn_date=normalize_statement_date(t.txn_date),
217                 particulars=t.particulars,
218                 merchant_category=t.merchant_category,
219                 amount=t.amount,
220                 type=t.type,
221             )
222             dbmod.record_parse(
223                 conn,
224                 doc_id,
225                 confidence=Confidence.UNVERIFIED.value,
226                 parser="AxisCreditCardParser",
227                 parser_version=__version__,
228             )
229         return len(rows)
230

```

7. user (2026-06-10T17:59:33.535Z)

```

1     """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.
2
3     This is the *generic-first* half of the concept-core architecture (see
4     [docs/parser-architecture.md](../docs/parser-architecture.md) §4, §6).
5     Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,

```

```

6      or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
7      via a cheap-first ladder, with a profile supplying priors:
8
9      * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
10     * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
11     * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
12         plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
13         reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
14         makes the math work* rather than by its label.
15
16     The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
17     names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
18     (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see
19     the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
20     value; provenance is the winning layer + column map.
21     ""
22
23     from __future__ import annotations
24
25     import itertools
26     import re
27     from collections.abc import Callable, Sequence
28     from dataclasses import dataclass, field
29
30     from .concepts import ConceptRow
31     from .dates import normalize_statement_date
32     from .validation import parse_amount
33
34     Cell = object # a table cell: str | None (we coerce defensively)
35     Row = Sequence[Cell]
36
37     # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
38     _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
39         re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"), # dd/mm/yyyy or dd-mm-yyyy
40         re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"), # dd-Mon-yyyy / dd Mon yy
41     )
42     # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
43     # for an amount column during content classification.
44     _MONEY_RE = re.compile(r"(?i)^\d{1,2}(?:\s*(?:cr|dr))?$")
45     _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")
46
47     # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
48     # lower-cased header labels matched exactly (not by substring) during L0.
49     _BANK_ALIASES: dict[str, tuple[str, ...]] = {
50         "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
51         "value_date": ("value date", "val date", "value dt"),
52         "narration": (
53             "narration",
54             "particulars",
55             "description",
56             "transaction details",
57             "details",
58             "remarks",
59             "transaction",

```

```

60         "transaction remarks",
61     ),
62     "reference": (
63         "ref",
64         "reference",
65         "ref no",
66         "reference no",
67         "cheque no",
68         "chq no",
69         "instrument no",
70     ),
71     "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
72     "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
73     "balance": (
74         "balance",
75         "closing balance",
76         "running balance",
77         "closing bal",
78         "available balance",
79     ),
80 }
81
82
83 def _text(cell: Cell) -> str:
84     return "" if cell is None else str(cell).strip()
85
86
87 def _norm(label: str) -> str:
88     return re.sub(r"\s+", " ", label.strip().lower())
89
90
91 def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
92     return any(p.match(text) for p in patterns)
93
94
95 def _looks_like_money(text: str) -> bool:
96     return bool(_MONEY_RE.match(text))
97
98
99 @dataclass
100 class FamilySpec:
101     """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.
102
103     The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
104     future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
105     ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
106     """
107
108     name: str
109     concepts: tuple[str, ...]
110     date_concepts: tuple[str, ...] # in order: the first is the primary (row-gating)
date
111     amount_concepts: tuple[str, ...] # money columns; the non-balance ones are the
"flows"
112     balance_concept: str | None # the running-balance concept, if the family has one
113     text_concepts: tuple[str, ...] # free-text columns; the first gets the widest text
column
114     default_aliases: dict[str, tuple[str, ...]]
115     build: Callable[[list[dict[str, str]]], list]
116
117

```

```

118     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
119         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
120         ``None``).
121         Dates are canonicalized to ISO-8601 here (day-first Indian formats; unrecognized
122         tokens
123         pass through verbatim) ? arithmetic cannot catch a date error, so at least the
124         stored
125         form is a validated calendar date.
126         """
127         rows: list[ConceptRow] = []
128         for m in mapped:
129             rows.append(
130                 ConceptRow(
131                     txn_date=normalize_statement_date(_text(m.get("txn_date"))) or "",
132                     narration=_text(m.get("narration")),
133                     debit=parse_amount(m.get("debit")),
134                     credit=parse_amount(m.get("credit")),
135                     # signed: a "Dr" suffix on a BALANCE means overdrawn (negative) ? see
136                     parse_amount
137                     balance=parse_amount(m.get("balance"), signed=True),
138                     value_date=normalize_statement_date(_text(m.get("value_date"))) or None,
139                     reference=_text(m.get("reference")) or None,
140                 )
141             )
142         return rows
143
144     BANK_FAMILY = FamilySpec(
145         name="bank",
146         concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
147         "balance"),
148         date_concepts=("txn_date", "value_date"),
149         amount_concepts=("debit", "credit", "balance"),
150         balance_concept="balance",
151         text_concepts=("narration", "reference"),
152         default_aliases=_BANK_ALIASES,
153         build=_build_bank_rows,
154     )
155
156     @dataclass
157     class MappingResult:
158         """The outcome of mapping one table: typed rows + how we got there (provenance)."""
159         rows: list # typed rows from family.build (e.g. list[ConceptRow])
160         mapped: list[
161             dict[str, str]
162         ] # per-row {concept: raw cell}, incl. any profile-declared extra columns
163         column_map: dict[str, int] # the winning concept -> column-index assignment
164         layer: str # "L0" / "profile-order" / "L2" ? which rung produced it
165         verified: bool # did the oracle endorse it? (False if no oracle was supplied)
166
167     @dataclass
168     class Profile:
169         """Declarative per-issuer knowledge ? *data, not code* (architecture §6).
170         Generic detectors run first; the profile supplies priors: extra concept?label
171         aliases, the
172         accepted date shape, a positional fallback when there's no header, table anchors,
173         account-number
174         patterns, quirks, and (placeholder, wiring parked) the password rule.
175         """

```

```

176     name: str
177     aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
178     date_patterns: tuple[re.Pattern, ...] = ()
179     column_order: tuple[str, ...] | None = None
180     table_anchors: tuple[str, ...] = ()
181     account_patterns: tuple[re.Pattern, ...] = ()
182     quirks: frozenset[str] = frozenset()
183     password_rule: str | None = None # data only; sourcing is parked (DESIGN §8.2)
184
185     def first_account(self, text: str) -> str | None:
186         for rx in self.account_patterns:
187             m = rx.search(text)
188             if m:
189                 return m.group(1).strip()
190         return None
191
192
193     def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
194         """Build ``normalized-label -> concept`` (profile entries override family
195 defaults)."""
196         index: dict[str, str] = {}
197         for concept, labels in family.default_aliases.items():
198             for label in labels:
199                 index[_norm(label)] = concept
200         for concept, labels in profile.aliases.items():
201             for label in labels:
202                 index[_norm(label)] = concept
203         return index
204
205     def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
206         return profile.date_patterns or _GENERIC_DATE_PATTERNS
207
208
209     def _detect_header(
210         rows: list[list[Cell]], alias_index: dict[str, str]
211     ) -> tuple[int | None, dict[str, int]]:
212         """Find the header row (most exact alias hits, >=2) and its concept->column map."""
213         best_idx: int | None = None
214         best_map: dict[str, int] = {}
215         for i, row in enumerate(rows):
216             cmap: dict[str, int] = {}
217             for col, cell in enumerate(row):
218                 concept = alias_index.get(_norm(_text(cell)))
219                 if concept and concept not in cmap:
220                     cmap[concept] = col
221             if len(cmap) > len(best_map):
222                 best_map, best_idx = cmap, i
223         if len(best_map) < 2:
224             return None, {}
225         return best_idx, best_map
226
227
228     def _is_balance_marker(row: Row) -> bool:
229         return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)
230
231
232     def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
233     -> bool:
234         if date_col is not None:
235             cell = row[date_col] if date_col < len(row) else None
236             return _matches_date(_text(cell), patterns)
237         return any(_matches_date(_text(c), patterns) for c in row)
238

```

```

239 def _classify_columns(
240     data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
241 ) -> dict[int, str]:
242     """Tag each column date / amount / text / empty by majority content."""
243     kinds: dict[int, str] = {}
244     for c in range(ncols):
245         cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
246         if not cells:
247             kinds[c] = "empty"
248             continue
249         thresh = max(1, int(len(cells) * 0.6))
250         if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
251             kinds[c] = "date"
252         elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
253             kinds[c] = "amount"
254         else:
255             kinds[c] = "text"
256     return kinds
257
258
259 def _base_map(
260     col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
261 ) -> dict[str, int]:
262     """The unambiguous part of a content (L1) mapping: date columns + the primary text
263     column.
264     Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
265     text column
266     goes to the family's first text concept (if any). Amount columns are left to the
267     search.
268     """
269     base: dict[str, int] = {}
270     date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
271     text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
272     for concept, col in zip(family.date_concepts, date_cols, strict=False):
273         base[concept] = col
274     if text_cols and family.text_concepts:
275         base[family.text_concepts[0]] = max(
276             text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
277         )
278     return base
279
280
281 def _search_candidates(
282     base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
283 ) -> list[tuple[str, dict[str, int]]]:
284     """Enumerate plausible amount-column ? concept assignments (the L2 search space).
285     Family-agnostic: the "flows" are the family's amount concepts minus its balance
286     concept (for
287     banks: debit, credit). We try each amount column as the balance and assign the rest
288     to the
289     flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
290     Oct).
291     """
292     amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
293     flows = [c for c in family.amount_concepts if c != family.balance_concept]
294     bal = family.balance_concept
295     cands: list[dict[str, int]] = []
296     if bal and len(amount_cols) >= len(flows) + 1:
297         for bcol in amount_cols:
298             rest = [c for c in amount_cols if c != bcol]
299             for assign in itertools.permutations(rest, len(flows)):
300                 cands.append(**base, **dict(zip(flows, assign, strict=True)), bal:
301                 bcol})

```

```

297         if flows and len(amount_cols) >= len(flows):
298             for assign in itertools.permutations(amount_cols, len(flows)):
299                 cand.append(**base, **dict(zip(flows, assign, strict=True)))
300         return [("L2", c) for c in cand[:24]]
301
302
303     def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
304         out: list[dict[str, str]] = []
305         for row in data:
306             out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
307 col_map.items()})
308         return out
309
310     def map_table(
311         table: Sequence[Row] | None, # Sequence (covariant) so list[list[str | None]]
312         callers fit
313         profile: Profile,
314         family: FamilySpec = BANK_FAMILY,
315         *,
316         oracle: Callable[[list], bool] | None = None,
317     ) -> MappingResult | None:
318         """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
319 candidates.
320 Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
321 With an
322 ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
323 the first
324 structurally-complete candidate is returned unverified (the caller reconciles
325 separately).
326 Returns ``None`` if no transaction rows are found, or the best unverified candidate
327 if an oracle
328 was supplied but nothing reconciled.
329 """
330 if not table:
331     return None
332 rows_in = [list(r) for r in table if r is not None]
333 if not rows_in:
334     return None
335 alias_index = _alias_index(profile, family)
336 patterns = _date_patterns(profile)
337
338 header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
339 start = header_idx + 1 if header_idx is not None else 0
340 date_col_hint = col_map_hdr.get("txn_date")
341 if date_col_hint is None and profile.column_order and "txn_date" in
342 profile.column_order:
343     date_col_hint = profile.column_order.index("txn_date")
344
345 data = [
346     r
347     for r in rows_in[start:]
348     if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
349 ]
350 if not data:
351     return None
352 ncols = max(len(r) for r in data)
353 col_kinds = _classify_columns(data, ncols, patterns)
354 base = _base_map(col_kinds, data, family)
355 flows = [c for c in family.amount_concepts if c != family.balance_concept]
356
357 candidates: list[tuple[str, dict[str, int]]] = []
358 if flows and all(c in col_map_hdr for c in flows): # L0: header pinned the flow
359     concepts

```

```

353         candidates.append(("L0", {**base, **col_map_hdr}))
354     elif col_map_hdr: # header present but didn't pin the flows: keep its hints, search
the rest
355         candidates.append(("L0", {**base, **col_map_hdr}))
356     if profile.column_order: # positional prior (no/partial header)
357         candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
358     candidates += _search_candidates(base, col_kinds, family)
359
360     primary_date = family.date_concepts[0] if family.date_concepts else None
361     best: MappingResult | None = None
362     for layer, cmap in candidates:
363         if primary_date and primary_date not in cmap:
364             continue
365         if not (set(family.amount_concepts) & cmap.keys()):
366             continue
367         mapped = _apply_map(data, cmap)
368         typed = family.build(mapped)
369         if oracle is None:
370             return MappingResult(typed, mapped, cmap, layer, verified=False)
371         if oracle(typed):
372             return MappingResult(typed, mapped, cmap, layer, verified=True)
373         if best is None:
374             best = MappingResult(typed, mapped, cmap, layer, verified=False)
375     return best
376
377
378 __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
379

```

8. user (2026-06-10T17:59:39.220Z)

```

1     """AU Small Finance Bank savings / transaction-account statement parser.
2
3     Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5     vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
entirely in
6     ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
here.
7     """
8
9     from __future__ import annotations
10
11     import sqlite3
12     from pathlib import Path
13
14     from muneem import __version__
15     from muneem import db as dbmod
16     from muneem.errors import ParseError
17     from muneem.redaction import redact_filename
18
19     from . import register_parser
20     from .base import StatementParser
21     from .profiles import AUBANK_PROFILE
22     from .savings import SavingsParse, parse_savings
23     from .validation import Confidence
24
25
26     def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
27         """Parse an AU Bank savings statement via the shared engine-driven path."""
28         return parse_savings(pdf_path, AUBANK_PROFILE, password)
29
30

```



```

31     @register_parser
32     class AUParser(StatementParser):
33         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
34             if sender and "aubank" in sender.lower():
35                 return True
36             low = (text or "").lower()
37             # Text evidence must be letterhead/footer-shaped: the spelled-out bank name, or
the
38             # bank's domain WITH its TLD ("aubank.in", as printed in statement footers). The
bare
39             # token "aubank" also appears in UPI handles ("UPI/someone@aubank/?") inside
OTHER
40             # banks' narrations ? and this parser is alphabetically first in the registry,
so a
41             # bare substring match hijacked routing for any statement that merely paid an AU
user.
42             return "au small finance bank" in low or "aubank.in" in low
43
44         def parse_and_insert(
45             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
46         ) -> int:
47             """Persist a savings parse **only if it reconciles**; otherwise store
nothing."""
48             try:
49                 parsed = parse_au_savings(pdf_path, password)
50             except ParseError:
51                 raise
52             except Exception as exc:
53                 raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc
54
55             if not parsed.verified:
56                 dbmod.record_parse(
57                     conn,
58                     doc_id,
59                     confidence=Confidence.UNRECONCILED.value,
60                     parser="AUParser",
61                     parser_version=__version__,
62                 )
63             return 0
64
65             for r in parsed.rows:
66                 dbmod.insert_au_transaction(
67                     conn,
68                     document_id=doc_id,
69                     account_number=parsed.account_number,
70                     txn_date=r.txn_date,
71                     value_date=r.value_date or "",
72                     particulars=r.narration,
73                     debit=r.debit,
74                     credit=r.credit,
75                     balance=r.balance,
76                 )
77             dbmod.record_parse(
78                 conn,
79                 doc_id,
80                 confidence=Confidence.RECONCILED.value,
81                 parser="AUParser",
82                 parser_version=__version__,
83             )
84             return len(parsed.rows)
85

```

9. user (2026-06-10T17:59:39.522Z)

```
1      """Coordinate clustering ? turn positioned words into a clean cell-matrix.
2
3      When ``pdfplumber.extract_tables()`` mis-groups a semi-ruled or whitespace-aligned
statement (the
4      Axis case: a date merged into the narration cell, column counts drifting between
vintages), we read
5      **word positions** directly. Words are grouped into visual rows by their ``top``; column
boundaries
6      are inferred from the vertical whitespace **shared across the transaction rows**; and
each word is
7      dropped into the column its x-centre falls in. Because columns come from geometry, a
leading date
8      sits in its *own* column instead of being glued to the narration.
9
10     Two robustness lessons baked in:
11
12     * **Infer columns from transaction-like rows only.** Headers, address blocks, and
summaries span the
13     page and would erase the inter-column whitespace. A caller-supplied ``keep_line``
predicate (e.g.
14     "has a date *and* a money token") restricts both column inference and the emitted
matrix to the
15     transaction band.
16     * **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel
empty
17     in *most* rows; a single long narration can't collapse two columns.
18
19     Bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's
oracle still
20     decides whether the result is trustworthy. Pure geometry in, strings out ? no value is
logged.
21     """
22
23     from __future__ import annotations
24
25     from collections.abc import Callable, Sequence
26
27     Word = dict # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
28
29
30     def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
31         """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-
right."""
32         lines: list[list[Word]] = []
33         anchor_top: float | None = None
34         for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
35             if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
36                 lines[-1].append(w)
37             else:
38                 lines.append([w])
39                 anchor_top = w["top"]
40         return [sorted(line, key=lambda w: w["x0"]) for line in lines]
41
42
43     def infer_columns(
44         lines: Sequence[Sequence[Word]], bin_w: float = 1.0, min_frac: float = 0.3
45     ) -> list[tuple[float, float]]:
46         """Infer column x-extents from a coverage histogram over ``lines``.
47
48         An x-bin is "filled" if at least ``min_frac`` of the lines have a word covering it;
columns are
49         the runs of filled bins, and the empty runs between them are the inter-column
whitespace. Robust
```

```

50         to a few full-width lines: it thresholds on the *fraction* of rows, not on any
single one.
51         """
52         words = [w for line in lines for w in line]
53         if not words:
54             return []
55         x0 = min(float(w["x0"]) for w in words)
56         x1 = max(float(w["x1"]) for w in words)
57         nbins = max(1, int((x1 - x0) / bin_w) + 1)
58         cover = [0] * nbins
59         for line in lines:
60             marked: set[int] = set()
61             for w in line:
62                 a = int((float(w["x0"]) - x0) / bin_w)
63                 b = int((float(w["x1"]) - x0) / bin_w)
64                 marked.update(range(max(0, a), min(nbins - 1, b) + 1))
65             for k in marked:
66                 cover[k] += 1
67         thresh = max(1.0, min_frac * len(lines))
68         filled = [c >= thresh for c in cover]
69         cols: list[tuple[float, float]] = []
70         start: int | None = None
71         for i, f in enumerate(filled):
72             if f and start is None:
73                 start = i
74             elif not f and start is not None:
75                 cols.append((x0 + start * bin_w, x0 + i * bin_w))
76                 start = None
77         if start is not None:
78             cols.append((x0 + start * bin_w, x0 + nbins * bin_w))
79         return cols
80
81
82     def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
83         for i, (a, b) in enumerate(columns):
84             if a <= center <= b:
85                 return i
86         return min(
87             range(len(columns)),
88             key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
89         )
90
91
92     def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) ->
list[str]:
93         """Place each word into its column; join words that share a column (in x order)."""
94         cells = ["" for _ in columns]
95         for w in line:
96             idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
97             cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
98         return cells
99
100
101     def cluster_to_matrix(
102         words_by_page: Sequence[Sequence[Word]],
103         *,
104         line_tol: float = 2.5,
105         min_frac: float = 0.3,
106         keep_line: Callable[[list[str]], bool] | None = None,
107     ) -> list[list[str]]:
108         """Cluster words across pages into one cell-matrix with consistent columns.
109
110         ``keep_line`` (texts of a line ? bool) restricts both column inference and the
output to the
111         transaction band ? without it, full-width headers/footers swamp the column

```

```

structure. Columns
112     are inferred once across all kept lines (a statement keeps its layout); rows are
emitted in
113     page-then-top order.
114     """
115     lines: list[list[Word]] = []
116     for page in words_by_page:
117         lines.extend(group_lines(page, line_tol))
118     if keep_line is not None:
119         lines = [ln for ln in lines if keep_line([str(w["text"]) for w in ln])]
120     if not lines:
121         return []
122     columns = infer_columns(lines, min_frac=min_frac)
123     if not columns:
124         return []
125     return [line_to_cells(line, columns) for line in lines]
126
127
128 __all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]
129

```

10. user (2026-06-10T17:59:44.556Z)

```

1     """The ``muneem`` command-line interface.
2
3     Track A ships the promo spine: ``ingest`` a PDF into the local store and
4     ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5     codes rasterized into banners (approximate; flagged for verification). Pipeline
6     subcommands (``fetch``, ``txns``) land with their milestones.
7     """
8
9     from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21
22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )
29     offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30     app.add_typer(offers_app, name="offers")
31
32     txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33     app.add_typer(txns_app, name="txns")
34
35     db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36     app.add_typer(db_app, name="db")
37
38
39     @contextmanager
40     def _ledger(db_path: Path):
41         """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI

```

```

42         error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43     try:
44         with dbmod.connect_encrypted(db_path) as conn:
45             yield conn
46     except dbmod.EncryptedDatabaseError as exc:
47         typer.echo(str(exc), err=True)
48         raise typer.Exit(1) from exc
49
50
51     # Dispatch table for supported banks (extensible as new parsers land).
52     BANK_DISPATCH = {
53         "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54         "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "amount"), # direction from
55         "type"
56         "au": (dbmod.list_au_transactions, "debit", "credit"),
57         "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
58     }
59
60     def _version_callback(value: bool) -> None:
61         if value:
62             typer.echo(f"muneem {__version__}")
63             raise typer.Exit()
64
65
66     @app.callback()
67     def main(
68         version: bool = typer.Option(
69             False,
70             "--version",
71             callback=_version_callback,
72             is_eager=True,
73             help="Show the muneem version and exit.",
74         ),
75     ) -> None:
76         """muneem command-line interface."""
77
78
79     @app.command()
80     def info() -> None:
81         """Show resolved local paths (no secrets, no sensitive data)."""
82         cfg = load_config()
83         present = "present" if cfg.config_file.is_file() else "absent"
84         typer.echo(f"muneem {__version__}")
85         typer.echo(f"config dir : {cfg.config_dir}")
86         typer.echo(f"data dir   : {cfg.data_dir}")
87         typer.echo(f"db path   : {cfg.db_path}")
88         typer.echo(f"staging  : {cfg.staging_dir}")
89         typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92     def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
93 bool) -> None:
94         """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
95
96         The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
97 extraction,
98 since the text needed to pick a parser is itself encrypted. The resolved password is
99 forwarded
100 to extraction and the parser, used in-memory only ? never logged.
101 """
102     from muneem import db as dbmod
103     from muneem.extract import extract
104     from muneem.parsers import get_parser
105     from muneem.statement_unlock import resolve_password

```

```

103     from muneem.unlock import is_encrypted
104
105     password: str | None = ""
106     if is_encrypted(pdf):
107         password = resolve_password(pdf, interactive=interactive)
108         if password is None:
109             hint = " (run without --no-prompt to enter it)" if not interactive else ""
110             typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
111             raise typer.Exit(1)
112
113     ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
str
114     parser = get_parser(ex.text)
115     if parser is None:
116         typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
117         raise typer.Exit(1)
118
119     with _ledger(db_path) as conn:
120         dbmod.init_db(conn)
121         existing = dbmod.get_document_by_sha(conn, ex.sha256)
122         if existing is not None and existing["status"] == "reconciled":
123             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
124             return
125         if existing is not None:
126             # sha256 dedup must not entomb a failure: a document the oracle did NOT
endorse
127             # gets a fresh parse (a parser fix deserves a second chance). Same document
id,
128             # clean row slate; provenance upserts on the re-parse.
129             doc_id = int(existing["id"])
130             dbmod.clear_document_rows(conn, doc_id)
131             typer.echo(
132                 f"re-parsing previously {existing['status']} document (doc {doc_id}):
{pdf.name}"
133             )
134         else:
135             doc_id = dbmod.insert_document(
136                 conn,
137                 source="file",
138                 external_id=pdf.name,
139                 doc_type=doc_type,
140                 sha256=ex.sha256,
141                 path=str(pdf.resolve()),
142                 status="parsed",
143             )
144             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
145             doc = dbmod.get_document_by_sha(conn, ex.sha256)
146             status = doc["status"] if doc else "parsed"
147
148         typer.echo(
149             f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
150             f"via {parser.__class__.__name__} [{status}]"
151         )
152
153
154     def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
155         """Run one statement through the full parse pipeline into an in-memory DB.
156
157         Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
status
158         (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
159         Nothing is persisted and no statement value is surfaced ? only the verdict and

```

```

counts.
160         """
161         from muneem import db as dbmod
162         from muneem.extract import extract
163         from muneem.parsers import get_parser
164         from muneem.statement_unlock import resolve_password
165         from muneem.unlock import is_encrypted
166
167         password = ""
168         if is_encrypted(pdf):
169             password = resolve_password(pdf, interactive=interactive) or ""
170             if not password:
171                 return ("locked", 0, None)
172         try:
173             ex = extract(pdf, password=password)
174             parser = get_parser(ex.text)
175         except Exception as exc: # eval must not crash on one bad file; record the redacted
reason
176             _log.warning(
177                 "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
178             )
179             return ("error", 0, None)
180         if parser is None:
181             return ("no-parser", 0, None)
182         name = parser.__class__.__name__
183         try:
184             with dbmod.connect(":memory:") as conn:
185                 dbmod.init_db(conn)
186                 doc_id = dbmod.insert_document(
187                     conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
188                 )
189                 count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
190                 doc = dbmod.get_document_by_sha(conn, ex.sha256)
191                 status = doc["status"] if doc else "parsed"
192         except Exception as exc:
193             _log.warning(
194                 "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
195             )
196             return ("error", 0, name)
197         return (status, count, name)
198
199
200     @app.command("eval")
201     def eval_statements(
202         folder: Path = typer.Argument(
203             ...,
204             exists=True,
205             file_okay=False,
206             readable=True,
207             help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
208         ),
209         prompt: bool = typer.Option(
210             False,
211             "--prompt/--no-prompt",
212             help="Prompt for a password when cached/configured candidates don't unlock a
file.",
213         ),
214     ) -> None:
215         """Evaluate the parsers against a folder of statements and report the reconcile
rate.
216

```

```

217         The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
218         statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
219         an in-memory DB; only counts / statuses / parser names print ? never statement
values.
220         """
221         pdfs = sorted(folder.glob("*.pdf"))
222         if not pdfs:
223             typer.echo(f"no .pdf files in {folder}")
224             return
225         reconciled = 0
226         for pdf in pdfs:
227             outcome, count, name = _eval_statement(pdf, interactive=prompt)
228             reconciled += outcome == "reconciled"
229             typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
230         typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
231
232
233     @app.command()
234     def ingest(
235         pdf: Path = typer.Argument(
236             ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
237         ),
238         doc_type: str = typer.Option("promo", help="Document category."),
239         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
240         ocr: bool = typer.Option(
241             False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
242         ),
243         no_prompt: bool = typer.Option(
244             False,
245             "--no-prompt",
246             help="Never prompt for a statement password; use only cached/configured
candidates.",
247         ),
248     ) -> None:
249         """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
250         from muneem.extract import extract
251         from muneem.offers import extract_offer
252
253         db_path = db or load_config().db_path
254
255         if doc_type == "bank_statement":
256             _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
257             return
258
259         from muneem import db as dbmod
260
261         ex = extract(pdf)
262         text_fields = extract_offer(ex.text)
263         # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
264         found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
265         expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
266
267         if ocr:
268             from muneem.ocr import ocr_pdf, tesseract_available
269
270             if not tesseract_available():
271                 typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
272             else:
273                 ocr_fields = extract_offer(ocr_pdf(pdf))
274                 known = {c for c, _ in found}
275                 for code in ocr_fields.codes:

```



```

276         if code not in known:
277             found.append((code, "ocr"))
278             known.add(code)
279         if expiry_raw is None:
280             expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
281
282     with _ledger(db_path) as conn:
283         dbmod.init_db(conn)
284         existing = dbmod.get_document_by_sha(conn, ex.sha256)
285         if existing is not None:
286             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
287             return
288         doc_id = dbmod.insert_document(
289             conn,
290             source="file",
291             external_id=pdf.name,
292             doc_type=doc_type,
293             sha256=ex.sha256,
294             path=str(pdf.resolve()),
295             status="parsed",
296         )
297         for code, src in found:
298             dbmod.insert_offer(
299                 conn,
300                 document_id=doc_id,
301                 promo_code=code,
302                 expiry_raw=expiry_raw,
303                 expiry_type=expiry_type,
304                 source=src,
305                 source_text=ex.text[:2000],
306             )
307
308     summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
309     typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
310
311
312 @offers_app.command("list")
313 def offers_list(
314     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
315     dir)."),
316     ) -> None:
317     """List extracted offers."""
318     from muneem import db as dbmod
319
320     db_path = db or load_config().db_path
321     if not Path(db_path).exists():
322         typer.echo("no offers yet (database not created).")
323         return
324
325     with _ledger(db_path) as conn:
326         dbmod.init_db(conn)
327         rows = dbmod.list_offers(conn)
328
329     if not rows:
330         typer.echo("no offers found.")
331         return
332
333     for row in rows:
334         who = row["merchant"] or row["external_id"] or "?"
335         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
336         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
337         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
338
339
340 @txns_app.command("list")
341 def txns_list(

```

```

340     bank: str = typer.Argument(
341         ..., help="Which bank to list (e.g. hdfc, axis) ? or 'all' for the cross-bank
view."
342     ),
343     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
344     limit: int = typer.Option(50, help="Max number of transactions to show."),
345 ) -> None:
346     """List parsed transactions for one bank, or across all banks via the serving
view."""
347     from muneem import db as dbmod
348
349     db_path = db or load_config().db_path
350     if not Path(db_path).exists():
351         typer.echo("database not created yet.")
352         return
353
354     bank = bank.lower()
355     if bank == "all":
356         with _ledger(db_path) as conn:
357             dbmod.init_db(conn)
358             rows = dbmod.list_all_transactions(conn, limit=limit)
359             if not rows:
360                 typer.echo("no transactions found.")
361                 return
362             for row in rows:
363                 amount = f"+{row['credit']}" if row["credit"] else f"-{row['debit']}"
364                 part = (row["particulars"] or "")[:40]
365                 verdict = row["confidence"] or "?"
366                 typer.echo(
367                     f"[{row['bank']}] : {row['account_number']} {row['txn_date']} : "
368                     f"{part:<40} | {amount:>10} [{verdict}]"
369                 )
370             return
371
372     if bank not in BANK_DISPATCH:
373         typer.echo(f"Unknown bank: {bank}", err=True)
374         return
375
376     list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
377     with _ledger(db_path) as conn:
378         dbmod.init_db(conn)
379         rows = list_fn(conn, limit=limit)
380
381     if not rows:
382         typer.echo(f"no {bank} transactions found.")
383         return
384     for row in rows:
385         if bank == "axis_cc":
386             sign = "+" if row["type"] == "Cr" else "-"
387             amount = f"{sign}{row['amount']}"
388         else:
389             amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
390             part = (row["particulars"] or "")[:50]
391             # Anything but a clean 'reconciled' verdict is surfaced, never silently listed.
392             status = row["doc_status"]
393             tag = "" if status == "reconciled" else f" [{status}]"
394             typer.echo(f"[{row['account_number']} {row['txn_date']} : {part:<50} |
{amount:>10}{tag}"]
395
396
397     @db_app.command("status")
398     def db_status(
399         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),

```

```

400 ) -> None:
401     """Show the ledger's encryption state (no statement data is read or printed)."""
402     from muneem import db as dbmod
403     from muneem.db_key import peek_key, peek_pending_key
404
405     db_path = db or load_config().db_path
406     key_state = "present in keychain" if peek_key() else "not yet minted"
407     if not Path(db_path).exists():
408         typer.echo(f"ledger : {db_path} (does not exist yet)")
409         typer.echo(f"key      : {key_state}")
410         return
411     if dbmod.is_plaintext_sqlite(db_path):
412         state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
413     else:
414         state = "encrypted (SQLCipher)"
415     typer.echo(f"ledger : {db_path}")
416     typer.echo(f"state  : {state}")
417     typer.echo(f"key    : {key_state}")
418     if peek_pending_key():
419         typer.echo(
420             "pending: a staged rotation key is present (an interrupted 'db rotate-key'?)
421             ? "
422             "the next ledger open reconciles it automatically."
423         )
424
425 @db_app.command("encrypt")
426 def db_encrypt(
427     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
428     dir)."),
429 ) -> None:
430     """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
431     place."""
432     from muneem import db as dbmod
433     from muneem.db_key import get_or_create_key
434
435     db_path = db or load_config().db_path
436     if not Path(db_path).exists():
437         typer.echo(
438             f"nothing to encrypt: {db_path} does not exist (a new ledger is born
439             encrypted)."
440         )
441         return
442     if not dbmod.is_plaintext_sqlite(db_path):
443         typer.echo(
444             f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
445             to do."
446         )
447         return
448     n = dbmod.encrypt_database(db_path, get_or_create_key())
449     typer.echo(
450         f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
451         remains at "
452         f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
453     )
454
455 def _require_encrypted_ledger(db_path: Path) -> str:
456     """Return the ledger's keychain key, or exit with a clear message if it can't be
457     used."""
458     from muneem import db as dbmod
459     from muneem.db_key import peek_key
460
461     if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
462         typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",

```

```

err=True)
458         raise typer.Exit(1)
459     key = peek_key()
460     if not key:
461         typer.echo("no encryption key in the keychain for this ledger.", err=True)
462         raise typer.Exit(1)
463     return key
464
465
466     @db_app.command("rotate-key")
467     def db_rotate_key(
468         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
469 dir)."),
470     ) -> None:
471         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
472         from muneem import db as dbmod
473         from muneem import db_key
474         from muneem.errors import KeychainError
475
476         db_path = db or load_config().db_path
477         current = _require_encrypted_ledger(db_path)
478         fresh = db_key.new_key()
479         # Both keys keep a durable copy at every instant: stage the fresh key under the
480 PENDING
481         # keychain entry, rekey the file, and only then promote it to the primary entry. Any
482         # interruption leaves a state that connect_encrypted can heal (file rekeyed ->
483 pending key
484         # promoted on next open; file untouched -> primary entry still matches it).
485         db_key.set_pending_key(fresh)
486         try:
487             dbmod.rekey_database(db_path, current, fresh)
488         except Exception as exc:
489             # Don't trust the exception to tell us how far the rekey got ? check the file.
490             if dbmod.key_opens_database(db_path, fresh):
491                 pass # the rekey actually completed; fall through to promotion below
492             elif dbmod.key_opens_database(db_path, current):
493                 db_key.clear_pending_key()
494                 typer.echo(
495                     f"key rotation failed; the ledger is unchanged (still under the existing
496 key): "
497                     f"{exc}",
498                     err=True,
499                 )
500                 raise typer.Exit(1) from exc
501             else:
502                 typer.echo(
503                     "key rotation failed and the ledger opens with neither key. Both keys
504 were kept "
505                     "in the keychain ('db-key' = old, 'db-key-pending' = new); restore the
506 ledger "
507                     "from a backup. Do NOT delete either keychain entry until the ledger
508 opens.",
509                     err=True,
510                 )
511                 raise typer.Exit(1) from exc
512             try:
513                 db_key.set_key(fresh) # promote: the file is verifiably under the fresh key now
514             except KeychainError as exc:
515                 typer.echo(
516                     "the ledger was rekeyed but promoting the new key in the keychain failed;
517 the new "
518                     "key is safe under 'db-key-pending' and the next ledger open will finish the
519
520                     "promotion automatically.",
521                     err=True,

```

```

513         )
514         raise typer.Exit(1) from exc
515     db_key.clear_pending_key()
516     typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
517     typer.echo(
518         "note: backups and any recovery file created BEFORE this rotation still use the
old "
519         "key. Re-run 'muneem db backup' and 'muneem db setup-recovery' now."
520     )
521
522
523     @db_app.command("backup")
524     def db_backup(
525         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
526         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
527     ) -> None:
528         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
529         from muneem import db as dbmod
530
531         db_path = db or load_config().db_path
532         key = _require_encrypted_ledger(db_path)
533         try:
534             dbmod.backup_database(db_path, dest, key)
535         except dbmod.EncryptedDatabaseError as exc:
536             typer.echo(str(exc), err=True)
537             raise typer.Exit(1) from exc
538         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
539
540
541     @db_app.command("setup-recovery")
542     def db_setup_recovery(
543         out: Path | None = typer.Option(
544             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
545         ),
546     ) -> None:
547         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
548
549         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
550         file holds only the *wrapped* key ? useless without the passphrase.
551         """
552         import getpass
553
554         from muneem import recovery
555         from muneem.db_key import peek_key
556
557         key = peek_key()
558         if not key:
559             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
560             raise typer.Exit(1)
561         out_path = out or (load_config().data_dir / "muneem-recovery.json")
562         passphrase = getpass.getpass("New recovery passphrase: ")
563         if passphrase != getpass.getpass("Confirm passphrase: "):
564             typer.echo("passphrases did not match.", err=True)
565             raise typer.Exit(1)
566         try:
567             recovery.write_recovery_file(out_path, key, passphrase)
568         except recovery.RecoveryError as exc:
569             typer.echo(str(exc), err=True)
570             raise typer.Exit(1) from exc
571         typer.echo(

```

```

572         f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "
573         "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
574         "lost keychain means the ledger and its backups are unrecoverable."
575     )
576
577
578     @db_app.command("recover")
579     def db_recover(
580         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
581         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
582     ) -> None:
583         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
584         import getpass
585
586         from muneem import db as dbmod
587         from muneem import db_key, recovery
588
589         passphrase = getpass.getpass("Recovery passphrase: ")
590         try:
591             key = recovery.read_recovery_file(file, passphrase)
592         except recovery.RecoveryError as exc:
593             typer.echo(str(exc), err=True)
594             raise typer.Exit(1) from exc
595         db_path = db or load_config().db_path
596         # If the ledger is present, verify the recovered key actually opens it before
committing it.
597         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
598             try:
599                 with dbmod.connect(db_path, key=key) as conn:
600                     conn.execute("SELECT 1")
601             except dbmod.EncryptedDatabaseError as exc:
602                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
603                 raise typer.Exit(1) from exc
604             db_key.set_key(key)
605             typer.echo("recovered the encryption key into the keychain.")
606
607
608     if __name__ == "__main__":
609         app()
610

```

11. user (2026-06-10T17:59:50.588Z)

```

1     """Resolve a statement PDF's password for the ingest path ? the hybrid strategy, wired.
2
3     DESIGN.md §8.2 / ROADMAP C.1. Resolution order when a statement is encrypted:
4
5     1. **Candidate dictionary** ? passwords cached in the OS keychain (from earlier
successful
6         unlocks) plus those configured in ``password_rules.yaml`` (see
7         :mod:`muneem.password_rules`).
8     2. **Interactive prompt** ? a hidden ``getpass`` prompt (skipped with
``interactive=False``,
9         e.g. automation / CI).
10
11     A password that works **via the prompt** is cached in the keychain so the next ingest is
12     friction-free. Nothing here logs a password, writes one to the repo, or echoes it; the
cache is
13     the OS keychain (:mod:`muneem.secrets`), never a file.

```

```

14     """
15
16     from __future__ import annotations
17
18     import getpass
19     import json
20     from pathlib import Path
21
22     from muneem import secrets
23     from muneem.errors import KeychainError
24     from muneem.password_rules import candidate_passwords
25     from muneem.unlock import is_encrypted, unlock
26
27     # Keychain entry holding a JSON list of known-good statement passwords (the learned
28     # candidates).
29     _CACHE_KEY = "statement_unlock_candidates"
30     # Bound the cached list so the keychain entry can't grow without limit; most-recently-
31     # used kept.
32     _MAX_CACHED = 20
33
34     def _cached_candidates() -> list[str]:
35         try:
36             raw = secrets.get_secret(_CACHE_KEY)
37         except KeychainError:
38             return [] # an unavailable keychain just means no cache this run
39         if not raw:
40             return []
41         try:
42             return [p for p in json.loads(raw) if isinstance(p, str) and p]
43         except (ValueError, TypeError):
44             return []
45
46     def _remember(password: str) -> None:
47         """Cache a working password in the keychain (MRU order, deduped, capped). Never
48         logged.
49
50         Best-effort: a keychain write failure is swallowed (the password already unlocked
51         the file this
52         run); we never crash an ingest over a caching hiccup.
53         """
54         if not password or not password.strip():
55             return
56         current = [p for p in _cached_candidates() if p != password]
57         updated = [password, *current][:_MAX_CACHED] # most-recently-used first, bounded
58         try:
59             secrets.set_secret(_CACHE_KEY, json.dumps(updated))
60         except KeychainError:
61             pass
62
63     def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
64         """Return a working password for ``pdf_path``.
65
66         ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is
67         encrypted and
68         no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next
69         time.
70         """
71         path = Path(pdf_path)
72         if not is_encrypted(path):
73             return ""
74
75         candidates = [*_cached_candidates(), *candidate_passwords()]

```

```

73     prompt_fn = None
74     if interactive:
75
76         def prompt_fn() -> str | None: # noqa: E306 ? local prompt; no echo, never
logged
77             return getpass.getpass(f"Password for {path.name}: ") or None
78
79     result = unlock(path, candidates=candidates, prompt_fn=prompt_fn)
80     if not result.ok:
81         return None
82     if result.method == "prompt" and result.password:
83         _remember(result.password)
84     return result.password or ""
85

```

12. user (2026-06-10T17:59:50.961Z)

```

1     """Password rule loader for statement decryption (hybrid strategy support).
2
3     Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
4     on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
5     dev) with a DeprecationWarning. **Never commit real rules or the file itself** ?
6     .gitignore`
7     blocks `password_rules.*`.
8
9     See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the
file
10    format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [...]``
list.
11    """
12    from __future__ import annotations
13
14    import warnings
15    from pathlib import Path
16    from typing import Any
17
18    import yaml
19
20    from muneem.config import CONFIG_DIR
21    from muneem.errors import ConfigError
22
23
24    def _load_rules() -> dict[str, Any]:
25        """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if
absent.
26
27        Resolution order:
28        1. ``<config_dir>/password_rules.yaml`` (recommended location)
29        2. ``./password_rules.yaml`` (legacy, emits DeprecationWarning)
30        """
31        for path, is_legacy in (
32            (CONFIG_DIR / "password_rules.yaml", False),
33            (Path("password_rules.yaml"), True),
34        ):
35            if path.exists():
36                if is_legacy:
37                    warnings.warn(
38                        "Loading password_rules from CWD (./password_rules.yaml) is
deprecated. "
39                        f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
proofing. "
40                        "See docs and CLAUDE.md (never commit real password material).",
41                        DeprecationWarning,
42                        stacklevel=3,

```



```

43         )
44         with path.open(encoding="utf-8") as f:
45             try:
46                 data = yaml.safe_load(f)
47             except yaml.YAMLError as exc:
48                 # Never echo the file's contents in the error ? they are password
material.
49                 raise ConfigError(
50                     f"could not parse password rules ({path.name}): invalid YAML"
51                 ) from exc
52             if data is None:
53                 return {}
54             if not isinstance(data, dict):
55                 raise ConfigError(
56                     f"password rules ({path.name}) must be a mapping of <bank>:
<password>"
57                 )
58             return data
59         return {}
60
61
62     def get_password_for_bank(bank_name: str) -> str | None:
63         """Return the password (or derived rule string) for a bank, or ``None``."""
64         val = _load_rules().get(bank_name)
65         return str(val) if val else None
66
67
68     def candidate_passwords() -> list[str]:
69         """Every configured password to try as an unlock candidate, **bank-agnostic**.
70
71         The ingest path resolves the password *before* it knows the issuer (it needs the
decrypted
72         text to pick a parser), so it tries all configured passwords. Collects each per-bank
value
73         plus any entries under a top-level ``candidates`` list; order preserved, de-
duplicated.
74         Quote numeric passwords in the YAML so leading zeros survive.
75         """
76         rules = _load_rules()
77         collected: list[str] = []
78         for key, val in rules.items():
79             if key == "candidates" and isinstance(val, list):
80                 collected.extend(str(v) for v in val if v not in (None, ""))
81             elif isinstance(val, str | int | float):
82                 collected.append(str(val))
83         seen: set[str] = set()
84         unique: list[str] = []
85         for pw in collected:
86             if pw and pw not in seen:
87                 seen.add(pw)
88                 unique.append(pw)
89         return unique
90

```

13. user (2026-06-10T17:59:56.010Z)

```

1     import pytest
2
3
4     @pytest.fixture(autouse=True)
5     def _isolate_keychain(monkeypatch):
6         """Never touch the real OS keychain in tests.
7
8         Replaces :mod:`muneem.secrets` with an in-memory store so the suite cannot pollute
the

```

```

9         developer's keychain (e.g. minting a DB key) and runs cleanly in headless CI where
no
10         keyring backend exists. Tests that need specific keychain behaviour re-patch on top.
11         """
12         store: dict[str, str] = {}
13
14         def _set(name, value):
15             store[name] = value
16
17         monkeypatch.setattr("muneem.secrets.get_secret", store.get)
18         monkeypatch.setattr("muneem.secrets.set_secret", _set)
19         monkeypatch.setattr("muneem.secrets.delete_secret", lambda name: store.pop(name,
None))
20         return store
21
22
23     # NOTE: the old `snapshot` fixture (testdata/snapshots/*.json) was removed deliberately.
24     # It wrote REAL parsed transaction rows to cleartext JSON inside the repo working dir ?
25     # the very data the encrypted ledger exists to protect ? and no test used it. If parse
26     # snapshotting returns, it must store only non-sensitive summaries (counts, reconcile
27     # booleans, hashes), never raw rows.
28

```

14. user (2026-06-10T17:59:56.620Z)

```

1     from pathlib import Path
2
3     from typer.testing import CliRunner
4
5     from muneem.cli import app
6     from muneem.extract import Extracted
7
8     runner = CliRunner()
9
10
11     def test_version_flag():
12         result = runner.invoke(app, ["--version"])
13         assert result.exit_code == 0
14         assert "muneem" in result.stdout
15
16
17     def test_info_lists_paths():
18         result = runner.invoke(app, ["info"])
19         assert result.exit_code == 0
20         assert "config dir" in result.stdout
21         assert "staging" in result.stdout
22
23
24     def test_ingest_bank_statement_unlocks_and_passes_password(tmp_path, monkeypatch):
25         """The bank path resolves the password and forwards it to extract + the parser."""
26         pdf = tmp_path / "stmt.pdf"
27         pdf.write_bytes(b"%PDF-1.4 fake")
28         captured = {}
29
30         monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: True)
31         monkeypatch.setattr(
32             "muneem.statement_unlock.resolve_password", lambda p, interactive=True: "PW123"
33         )
34
35         def fake_extract(p, password=""):
36             captured["extract_pw"] = password
37             return Extracted(path=Path(p), sha256="sha-x", text="Axis Bank Account
Statement")
38
39         monkeypatch.setattr("muneem.extract.extract", fake_extract)

```

```

40
41     class FakeParser:
42         def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
43             captured["parser_pw"] = password
44             return 3
45
46     monkeypatch.setattr("muneem.parsers.get_parser", lambda text: FakeParser())
47
48     result = runner.invoke(
49         app, ["ingest", str(pdf), "--doc-type", "bank_statement", "--db", str(tmp_path /
50 "t.db")]
51     )
52     assert result.exit_code == 0, result.output
53     assert captured["extract_pw"] == "PW123" and captured["parser_pw"] == "PW123"
54     assert "ingested" in result.output and "FakeParser" in result.output
55
56     def test_quarantined_document_can_be_reingested_after_parser_fix(tmp_path, monkeypatch):
57         """sha256 dedup must not entomb a failed parse: unreconciled docs get a second
58 chance,
59 reconciled docs stay deduped."""
60         from muneem import db as dbmod
61         from muneem.parsers.validation import Confidence
62
63         pdf = tmp_path / "stmt.pdf"
64         pdf.write_bytes(b"%PDF-1.4 fake")
65         db = tmp_path / "t.db"
66         monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: False)
67         monkeypatch.setattr(
68             "muneem.extract.extract",
69             lambda p, password="": Extracted(
70                 path=Path(p), sha256="sha-q", text="Axis Bank Account Statement"
71             ),
72         )
73
74         class BrokenParser: # the pre-fix parser: quarantines, stores nothing
75             def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
76                 dbmod.record_parse(
77                     conn, doc_id, confidence=Confidence.UNRECONCILED.value,
78                     parser="BrokenParser"
79                 )
80                 return 0
81
82         class FixedParser: # after the parser fix: stores a row and reconciles
83             def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
84                 dbmod.insert_axis_transaction(
85                     conn,
86                     document_id=doc_id,
87                     account_number="AX1",
88                     txn_date="01/01/2026",
89                     value_date="01/01/2026",
90                     particulars="recovered row",
91                     debit=100.0,
92                     credit=None,
93                     balance=900.0,
94                 )
95                 dbmod.record_parse(
96                     conn, doc_id, confidence=Confidence.RECONCILED.value,
97                     parser="FixedParser"
98                 )
99                 return 1
100
101     args = ["ingest", str(pdf), "--doc-type", "bank_statement", "--db", str(db)]
102     monkeypatch.setattr("muneem.parsers.get_parser", lambda text: BrokenParser())

```

```

101     first = runner.invoke(app, args)
102     assert first.exit_code == 0, first.output
103     assert "[unreconciled]" in first.output
104
105     monkeypatch.setattr("muneem.parsers.get_parser", lambda text: FixedParser())
106     second = runner.invoke(app, args)
107     assert second.exit_code == 0, second.output
108     assert "re-parsing previously unreconciled" in second.output
109     assert "[reconciled]" in second.output
110
111     third = runner.invoke(app, args) # now trusted -> dedup holds
112     assert third.exit_code == 0, third.output
113     assert "already ingested" in third.output
114
115     with dbmod.connect_encrypted(db) as conn:
116         doc = dbmod.get_document_by_sha(conn, "sha-q")
117         assert doc["status"] == "reconciled"
118         rows = dbmod.list_axis_transactions(conn, limit=10)
119         assert len(rows) == 1 # the re-parse started from a clean slate (no duplicates)
120
121
122 def test_db_status_on_missing_ledger(tmp_path):
123     result = runner.invoke(app, ["db", "status", "--db", str(tmp_path / "nope.db")])
124     assert result.exit_code == 0, result.output
125     assert "does not exist yet" in result.output
126     assert "not yet minted" in result.output
127
128
129 def test_db_encrypt_migrates_and_status_reflects_it(tmp_path):
130     import sqlite3
131
132     p = tmp_path / "ledger.db"
133     con = sqlite3.connect(p)
134     con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
135     con.execute("INSERT INTO t (amount) VALUES (10.0)")
136     con.commit()
137     con.close()
138
139     before = runner.invoke(app, ["db", "status", "--db", str(p)])
140     assert before.exit_code == 0 and "PLAINTEXT" in before.output
141
142     enc = runner.invoke(app, ["db", "encrypt", "--db", str(p)])
143     assert enc.exit_code == 0, enc.output
144     assert "migrated" in enc.output and "plaintext.bak" in enc.output
145
146     after = runner.invoke(app, ["db", "status", "--db", str(p)])
147     assert after.exit_code == 0 and "encrypted" in after.output
148
149
150 def test_ingest_bank_statement_unlock_failure_exits_nonzero(tmp_path, monkeypatch):
151     pdf = tmp_path / "stmt.pdf"
152     pdf.write_bytes(b"%PDF-1.4 fake")
153     monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: True)
154     monkeypatch.setattr(
155         "muneem.statement_unlock.resolve_password", lambda p, interactive=True: None
156     )
157     result = runner.invoke(
158         app,
159         [
160             "ingest",
161             str(pdf),
162             "--doc-type",
163             "bank_statement",
164             "--db",
165             str(tmp_path / "t.db"),

```

```

166         "--no-prompt",
167     ],
168 )
169     assert result.exit_code == 1
170     assert "Could not unlock" in result.output
171

```

15. user (2026-06-10T18:00:02.485Z)

```

1     """HDFC savings / transaction-account statement parser.
2
3     **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4     default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5     table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6     decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7     position. pdfplumber's whole chain is permissive and it opens password-protected
8     statements directly.
9
10    **Layout.** Each transaction is one *primary* line ? the transaction date at the
11    far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12    Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13    narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14    cheque/reference number. We group words into visual lines by ``top``, detect
15    primary lines by a leading date + a closing-balance amount, attach continuations,
16    and read columns by x-position.
17
18    **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19    balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20    ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21    statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22    parse complete without inspecting any individual value ? which is how we validate
23    against real (sensitive) statements. No statement content is logged here.
24    """
25
26    from __future__ import annotations
27
28    import csv
29    import re
30    from dataclasses import dataclass, field
31    from pathlib import Path
32
33    import pdfplumber
34
35    from muneem import __version__
36    from muneem import db as dbmod
37    from muneem.errors import ParseError
38    from muneem.redaction import redact_filename
39
40    from . import register_parser
41    from .base import StatementParser
42    from .concepts import ConceptRow, StatementConcepts
43    from .dates import normalize_statement_date
44    from .profiles import HDFC_PROFILE
45    from .validation import (
46        BALANCE_TOLERANCE,
47        BookendReport,
48        Confidence,
49        ReconcileReport,
50        reconcile_running_balance,
51        verify,
52    )
53
54    # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
55    # Left-aligned columns key on x0; right-aligned amount columns key on their right
56    # edge (x1), which is stable regardless of the number's magnitude.

```

```

57     _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
58     _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
59     _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
60     _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
61     _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
62     _LINE_TOL = 2.5 # words within this many points of each other share a visual line
63     _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
64
65     _DATE_RE = re.compile(r"^\d{2}/\d{2}/\d{4}$")
66     # Indian-format money: optional minus, comma groups, exactly two decimals, optional
Cr/Dr.
67     _AMOUNT_RE = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
68     _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
69
70
71     @dataclass
72     class Transaction:
73         date: str # dd/mm/yyyy, as printed
74         narration: str
75         withdrawal: float
76         deposit: float
77         closing_balance: float
78         value_date: str | None = None
79         ref: str | None = None
80         page: int = 0
81
82         def to_concept(self) -> ConceptRow:
83             """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
verifier's input).
84
85             Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
``page``
86             field is HDFC-internal provenance and has no concept equivalent.
87             """
88             return ConceptRow(
89                 txn_date=self.date,
90                 narration=self.narration,
91                 debit=self.withdrawal,
92                 credit=self.deposit,
93                 balance=self.closing_balance,
94                 value_date=self.value_date,
95                 reference=self.ref,
96             )
97
98
99     @dataclass
100     class Statement:
101         path: Path
102         transactions: list[Transaction] = field(default_factory=list)
103         account_number: str | None = None
104
105
106     # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
vocabulary +
107     # verifier (the concept-core refactor). These back-compat names keep HDFC's public
surface stable:
108     # StatementSummary == the statement-level concepts a SUMMARY block supplies
109     # ValidationReport == the verifier's bookend report (per-row walk + printed-figure
checks)
110     StatementSummary = StatementConcepts
111     ValidationReport = BookendReport
112
113
114     def _to_amount(token: str) -> float | None:
115         """Parse an HDFC money token to a float, or None if it is not money.

```

```

116
117     Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
118     is real information and the reconciliation math needs it.
119     """
120     if not _AMOUNT_RE.match(token):
121         return None
122     cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")
123     try:
124         return float(cleaned)
125     except ValueError:
126         return None
127
128
129 def _group_lines(words: list[dict]) -> list[list[dict]]:
130     """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
131     lines: list[list[dict]] = []
132     anchor_top: float | None = None
133     for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
134         if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
135             lines[-1].append(w)
136         else:
137             lines.append([w])
138             anchor_top = w["top"]
139     return [sorted(line, key=lambda w: w["x0"]) for line in lines]
140
141
142 def _line_top(line: list[dict]) -> float:
143     return min(w["top"] for w in line)
144
145
146 def _has_narration(line: list[dict]) -> bool:
147     return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
148
149
150 def _has_amount(line: list[dict]) -> bool:
151     return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
152
153
154 def _has_leading_date(line: list[dict]) -> bool:
155     first = line[0]
156     return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
157
158
159 def _has_footer_marker(line: list[dict]) -> bool:
160     return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
161
162
163 def _is_primary(line: list[dict]) -> bool:
164     """A transaction's first line: a date at the far left + a closing-balance amount.
165
166     Requiring the closing-balance amount guards against stray dates in the page
167     header / account-summary box (e.g. the statement period).
168     """
169     if not _has_leading_date(line):
170         return False
171     return any(
172         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
173         _AMOUNT_RE.match(w["text"])
174         for w in line
175     )
176
177 def _is_continuation(line: list[dict]) -> bool:
178     """A narration-wrap line: narration-band text, no leading date, no amounts."""
179     return _has_narration(line) and not _has_leading_date(line) and not

```

```

_has_amount(line)
180
181
182 def _parse_transaction(
183     primary: list[dict], continuations: list[list[dict]], page_no: int
184 ) -> Transaction:
185     withdrawal = deposit = 0.0
186     closing: float | None = None
187     narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
188
189     for line in (primary, *continuations):
190         for w in line:
191             x0, x1, text = w["x0"], w["x1"], w["text"]
192             if x0 >= _AMOUNT_MIN_X0:
193                 val = _to_amount(text)
194                 if val is None:
195                     continue
196                 if x1 < _WITHDRAWAL_MAX_X1:
197                     withdrawal = val
198                 elif x1 < _DEPOSIT_MAX_X1:
199                     deposit = val
200                 else:
201                     closing = val
202             elif x0 >= _NARR_MIN_X0:
203                 narr.append((w["top"], x0, text))
204
205     value_date: str | None = None
206     parts: list[str] = []
207     for _top, _x0, text in sorted(narr):
208         if value_date is None and _DATE_RE.match(text):
209             value_date = text # the value date, lifted out of the narration band
210             continue
211         parts.append(text)
212
213     ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
214     ref = max(ref_candidates, key=len) if ref_candidates else None
215
216     return Transaction(
217         date=primary[0]["text"],
218         narration=" ".join(parts),
219         withdrawal=withdrawal,
220         deposit=deposit,
221         closing_balance=closing if closing is not None else 0.0,
222         value_date=value_date,
223         ref=ref,
224         page=page_no,
225     )
226
227
228 def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
229     """Pure, PDF-free core: turn grouped table-body lines into transactions.
230
231     Kept separate from PDF I/O so the column logic is unit-testable on synthetic
232     word lists, with no real data.
233     """
234     primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
235     transactions: list[Transaction] = []
236     for k, idx in enumerate(primaries):
237         stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
238         primary = lines[idx]
239         primary_top = _line_top(primary)
240         continuations: list[list[dict]] = []
241         for m in range(idx + 1, stop):
242             line = lines[m]

```



```

243         if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
244             break
245             if _is_continuation(line):
246                 continuations.append(line)
247                 transactions.append(_parse_transaction(primary, continuations, page_no))
248     return transactions
249
250
251 def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
252     """Lines below the column header (drops the per-page account-summary box)."""
253     header_top: float | None = None
254     for line in lines:
255         texts = {w["text"] for w in line}
256         if "Narration" in texts and any(t.startswith("Closing") for t in texts):
257             header_top = _line_top(line)
258             break
259     if header_top is None:
260         return []
261     return [line for line in lines if _line_top(line) > header_top + 1]
262
263
264 def _extract_account_number(text: str) -> str | None:
265     """Account number via the HDFC profile (bank knowledge as data; same pattern as
before)."""
266     return HDFC_PROFILE.first_account(text)
267
268
269 def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
270     """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
271
272     ``password`` is forwarded to pdfplumber for encrypted statements and used
273     in-memory only ? never stored or logged.
274     """
275     path = Path(pdf_path)
276     transactions: list[Transaction] = []
277     account_number: str | None = None
278     with pdfplumber.open(str(path), password=password or "") as pdf:
279         for page_no, page in enumerate(pdf.pages):
280             if account_number is None and page_no < 2:
281                 account_number = _extract_account_number(page.extract_text() or "")
282                 words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
283                 body = _table_body(_group_lines(words))
284                 transactions.extend(extract_transactions_from_lines(body, page_no))
285     return Statement(path=path, transactions=transactions,
account_number=account_number)
286
287
288 def reconcile(statement: Statement, tol: float = BALANCE_TOLERANCE) -> ReconcileReport:
289     """Check the running balance via the shared reconciler (closing[i] ==
290     closing[i-1] - withdrawal + deposit).
291
292     Spans page breaks (transactions are in document order), so a clean report also
293     validates page stitching. Reports counts only ? no amounts.
294     """
295     entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
statement.transactions]
296     return reconcile_running_balance(entries, tol)
297
298
299 def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
300     """Pure core: read HDFC's SUMMARY block from extracted text lines.
301
302     Layout (stable across statements)::
303

```

```

304         SUMMARY
305         Opening Balance  Debit Amount  Credit Amount  Closing Balance
306         <opening>      <debitAmt>    <creditAmt>    <closing>
307         Debit Count  Credit Count
308         <debitCount> <creditCount>
309
310     The values land on their own lines, so we take the first post-SUMMARY line
311     carrying ?4 money tokens (the balances) and the first carrying ?2 bare
312     integers and no money (the counts).
313     """
314     start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
315     if start is None:
316         return None
317     balances: list[float] | None = None
318     counts: list[int] | None = None
319     for ln in lines[start + 1 : start + 8]:
320         tokens = ln.split()
321         monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
322         ints = [int(t) for t in tokens if t.isdigit()]
323         if balances is None and len(monies) >= 4:
324             balances = monies[:4]
325         elif counts is None and not monies and len(ints) >= 2:
326             counts = ints[:2]
327     if balances is None and counts is None:
328         return None
329     bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
330     opening, debit_amt, credit_amt, closing = bvals[:4]
331     cvals: list[int | None] = [*counts] if counts else [None, None]
332     debit_count, credit_count = cvals[:2]
333     return StatementSummary(
334         opening_balance=opening,
335         closing_balance=closing,
336         debit_count=debit_count,
337         credit_count=credit_count,
338         debit_amount=debit_amt,
339         credit_amount=credit_amt,
340     )
341
342
343     def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
344         """Read the bank's own SUMMARY figures from the statement, or None if absent."""
345         with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
346             for page in pdf.pages:
347                 text = page.extract_text() or ""
348                 if "SUMMARY" in text:
349                     summary = _parse_summary_lines(text.splitlines())
350                     if summary is not None:
351                         return summary
352             return None
353
354
355     def validate_against_summary(
356         statement: Statement, summary: StatementSummary, tol: float = BALANCE_TOLERANCE
357     ) -> ValidationReport:
358         """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
359
360         Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
361         (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
362         error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
363         closing
364         balances catch a missing first/last row.
365         """
366         rows = [t.to_concept() for t in statement.transactions]
367         return verify(rows, summary, tol)

```

```

368
369 def write_csv(statement: Statement, out_path: Path | str) -> Path:
370     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
371
372     Writes real data ? call only for a local, gitignored destination.
373     """
374     out = Path(out_path)
375     out.parent.mkdir(parents=True, exist_ok=True)
376     with open(out, "w", newline="", encoding="utf-8") as fh:
377         writer = csv.writer(fh)
378         writer.writerow(
379             [
380                 "date",
381                 "value_date",
382                 "narration",
383                 "ref",
384                 "withdrawal",
385                 "deposit",
386                 "closing_balance",
387                 "page",
388             ]
389         )
390         for t in statement.transactions:
391             writer.writerow(
392                 [
393                     t.date,
394                     t.value_date or "",
395                     t.narration,
396                     t.ref or "",
397                     f"{t.withdrawal:.2f}",
398                     f"{t.deposit:.2f}",
399                     f"{t.closing_balance:.2f}",
400                     t.page,
401                 ]
402             )
403         return out
404
405
406 @register_parser
407 class HDFCParser(StatementParser):
408     """Registered parser for HDFC savings / transaction-account statements."""
409
410     def can_handle(self, text: str, sender: str | None = None, subject: str | None =
411 None) -> bool:
412         low = (text or "").lower()
413         # Text evidence is the spelled-out letterhead name or the domain with its TLD ?
414         the
415         # bare "hdfcbank" token also appears in UPI handles ("?@hdfcbank") inside other
416         # banks' narrations and must not contribute to routing. Sender metadata stays
417         exempt
418         # (a narration cannot fake the sender).
419         is_hdfc = (
420             "hdfc bank" in low or "hdfcbank.com" in low or "hdfcbank" in (sender or
421 "").lower()
422         )
423         is_statement = "savings account details" in low or (
424             "narration" in low and "closing balance" in low
425         )
426         return is_hdfc and is_statement
427
428     def parse_and_insert(
429         self,
430         pdf_path: Path,
431         conn,
432         doc_id: int,

```

```

429         password: str | None = None,
430     ) -> int:
431         """Persist a parse **only if it reconciles**; otherwise store nothing.
432
433         Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
434         balance walk (and the bank's SUMMARY when present). An unreconciled parse is
435         quarantined ? status recorded, zero rows persisted ? matching the shared
436         savings path (Axis/AU): the ledger never holds rows the oracle didn't endorse.
437         Returns the persisted row count.
438         """
439         try:
440             stmt = parse_statement(pdf_path, password=password or "")
441         except ParseError:
442             raise
443         except Exception as exc:
444             raise ParseError(f"failed to parse HDFC statement
445 {redact_filename(pdf_path)}") from exc
446         summary = parse_summary(pdf_path, password=password or "")
447         ok = (
448             validate_against_summary(stmt, summary).ok
449             if summary is not None
450             else reconcile(stmt).ok
451         )
452         if not ok:
453             dbmod.record_parse(
454                 conn,
455                 doc_id,
456                 confidence=Confidence.UNRECONCILED.value,
457                 parser="HDFCParser",
458                 parser_version=__version__,
459             )
460             return 0
461         for t in stmt.transactions:
462             dbmod.insert_hdfc_transaction(
463                 conn,
464                 document_id=doc_id,
465                 account_number=stmt.account_number,
466                 txn_date=normalize_statement_date(t.date),
467                 value_date=normalize_statement_date(t.value_date),
468                 particulars=t.narration,
469                 ref=t.ref,
470                 withdrawal=t.withdrawal,
471                 deposit=t.deposit,
472                 closing_balance=t.closing_balance,
473             )
474         dbmod.record_parse(
475             conn,
476             doc_id,
477             confidence=Confidence.RECONCILED.value,
478             parser="HDFCParser",
479             parser_version=__version__,
480         )
481         return len(stmt.transactions)

```

16. user (2026-06-10T18:00:03.090Z)

```

1         """The **verifier** ? reconciliation as a layout-independent truth oracle.
2
3         Reconciliation proves a parse is **complete and correct without inspecting any sensitive
4         value**:
5         if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
6         us trust a
7         parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
8         safe on

```

```

6      money ? *extraction proposes, reconciliation disposes* (see
7      [docs/parser-architecture.md](../docs/parser-architecture.md) §2, §5).
8
9      This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit
+ credit``.
12      * **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing``
(work even
13      with no per-row balance, e.g. Axis savings).
14      * **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
16      endorses (the seam the L2 mapping search plugs into).
17      * **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict,
not an
18      assumption.
19
20      Reports carry counts / booleans / a verdict only ? **never an amount**.
21      ""
22
23      from __future__ import annotations
24
25      import re
26      from collections.abc import Sequence
27      from dataclasses import dataclass
28      from enum import StrEnum
29
30      from .concepts import ConceptRow, StatementConcepts
31
32      _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34      # Statement money is printed to two decimals, so the smallest REAL discrepancy is one
paisa
35      # (0.01); the walk's float arithmetic accumulates noise many orders of magnitude below
that.
36      # The acceptance tolerance therefore sits at HALF the quantum: genuine float noise
passes,
37      # while an exact one-paisa corruption always fails. (With tol == 0.01 and a strict-
greater
38      # compare, a measured ~40% of exact one-paisa corruptions used to land inside the
tolerance.)
39      BALANCE_TOLERANCE = 0.005
40
41
42      def parse_amount(value: object, *, signed: bool = False) -> float | None:
43          """Parse a statement money cell to a float.
44
45          Returns ``None`` only for a genuinely absent cell (None / empty / ``"-"" /
non-numeric); ``"0.00"" parses to ``0.0``. This deliberately distinguishes
46          *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"" to
47          ``None`` and would corrupt a real zero balance and the reconciliation math.
48
49
50          A trailing ``Cr``/``Dr`` marker is interpreted per column kind:
51
52          - ``signed=False`` (default, for **flow** cells): the marker is a *direction* tag;
the
53          magnitude is returned and direction lives in the debit-vs-credit assignment.
54          - ``signed=True`` (for **balance** cells): the marker is a *sign* ? ``Dr`` means the
55          account is overdrawn (negative), ``Cr`` in credit (positive). Stripping the sign
off
56          ``Dr`` balances lets a fully direction-inverted parse of an overdraft statement
57          reconcile, which is the worst possible verifier failure.

```

```

58     """
59     if value is None:
60         return None
61     raw = str(value).strip()
62     suffix = _CR_DR_SUFFIX.search(raw)
63     text = _CR_DR_SUFFIX.sub("", raw).replace(",", "").strip()
64     if text in ("", "-"):
65         return None
66     try:
67         amount = float(text)
68     except ValueError:
69         return None
70     if signed and suffix:
71         amount = -abs(amount) if suffix.group(1).lower() == "dr" else abs(amount)
72     return amount
73
74
75 @dataclass
76 class ReconcileReport:
77     """Result of a running-balance walk. Counts only, never an amount."""
78
79     n: int # rows considered
80     compared: int # consecutive (balance-bearing) pairs actually checked
81     breaks: int
82     first_break: int | None = None
83
84     @property
85     def ok(self) -> bool:
86         return self.compared > 0 and self.breaks == 0
87
88
89 def reconcile_running_balance(
90     entries: Sequence[tuple[float | None, float | None, float | None]],
91     tol: float = BALANCE_TOLERANCE,
92 ) -> ReconcileReport:
93     """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
94
95     ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
96     ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
97     is skipped (can't anchor on it). Because the walk spans the whole statement, a
98     clean result also rules out any row missed *between* two balances ? which is the
99     failure mode silent ``extract_tables()`` row-drops would otherwise hide.
100     """
101     breaks = 0
102     first_break: int | None = None
103     compared = 0
104     prev: float | None = None
105     for i, (debit, credit, balance) in enumerate(entries):
106         if balance is None:
107             continue
108         if prev is not None:
109             expected = prev - (debit or 0.0) + (credit or 0.0)
110             compared += 1
111             if abs(expected - balance) > tol:
112                 breaks += 1
113                 if first_break is None:
114                     first_break = i
115         prev = balance
116     return ReconcileReport(
117         n=len(entries), compared=compared, breaks=breaks, first_break=first_break
118     )
119
120
121 def _row_entries(
122     rows: Sequence[ConceptRow],

```

```

123     ) -> list[tuple[float | None, float | None, float | None]]:
124         """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
consumes."""
125         return [(r.debit, r.credit, r.balance) for r in rows]
126
127
128     def reconcile_total(
129         rows: Sequence[ConceptRow], opening: float, closing: float, tol: float =
BALANCE_TOLERANCE
130     ) -> bool:
131         """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.
132
133         This is the oracle for statements with no reliable per-row balance (e.g. Axis
savings):
134         it checks the whole statement against its own opening/closing bookends without
needing each
135         row's running balance. ``opening`` and ``closing`` come from the bank's
SUMMARY/header
136         (``StatementConcepts``); the caller must have them.
137         """
138         delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
139         return abs((opening + delta) - closing) <= tol
140
141
142     class Confidence(StrEnum):
143         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).
144
145         ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
146         a check failed (or the document family has checks but none could run); the parse is
suspect
147         and is quarantined rather than trusted. ``UNVERIFIED`` ? the document family offers
no
148         arithmetic oracle by design** (e.g. a credit-card bill with no running balance):
rows are
149         persisted but explicitly flagged, never presented as trusted. The listing layer
surfaces
150         everything that is not cleanly ``RECONCILED``.
151         """
152
153         RECONCILED = "reconciled"
154         UNRECONCILED = "unreconciled"
155         UNVERIFIED = "unverified"
156
157
158     @dataclass
159     class BookendReport:
160         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
161
162         ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
163         requires the per-row balance walk plus every available check to pass.
164         """
165
166         n: int
167         reconciles: bool
168         debit_count_ok: bool | None = None
169         credit_count_ok: bool | None = None
170         opening_ok: bool | None = None
171         closing_ok: bool | None = None
172
173     @property
174     def ok(self) -> bool:
175         checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,

```

```

self.closing_ok]
176         return self.n > 0 and self.reconciles and all(c is True for c in checks)
177
178     @property
179     def confidence(self) -> Confidence:
180         return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
181
182
183     def verify(
184         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float =
BALANCE_TOLERANCE
185     ) -> BookendReport:
186         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.
187
188         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
189         extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
190         figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
191         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
192         """
193         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
194         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
195         if concepts.debit_count is not None:
196             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
197         if concepts.credit_count is not None:
198             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
199         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
200             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
201         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
202             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
203             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
204         return BookendReport(
205             n=len(rows),
206             reconciles=reconciles,
207             debit_count_ok=debit_count_ok,
208             credit_count_ok=credit_count_ok,
209             opening_ok=opening_ok,
210             closing_ok=closing_ok,
211         )
212
213
214     def select_reconciling(
215         candidates: Sequence[Sequence[ConceptRow]], tol: float = BALANCE_TOLERANCE
216     ) -> int | None:
217         """Among candidate concept-row mappings, return the index of the first that
reconciles.
218
219         This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
220         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
221         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
222
223         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
224         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
225         running balance ? balance-less statements select via :func:`reconcile_total` at the

```



```

call site.
226     """
227     for i, rows in enumerate(candidates):
228         if reconcile_running_balance(_row_entries(rows), tol).ok:
229             return i
230     return None
231
232
233     __all__ = [
234         "BALANCE_TOLERANCE",
235         "parse_amount",
236         "ReconcileReport",
237         "reconcile_running_balance",
238         "reconcile_total",
239         "Confidence",
240         "BookendReport",
241         "verify",
242         "select_reconciling",
243     ]
244

```

17. user (2026-06-10T18:00:12.294Z)

```

1     """Decrypt password-protected statement PDFs (Stage 2).
2
3     Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
4     password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
5     fall back to an interactive prompt. The working password is returned so the
6     caller can cache it in the OS keychain (muneem.secrets); it is never logged.
7     """
8
9     from __future__ import annotations
10
11     import shutil
12     from collections.abc import Callable, Iterable
13     from dataclasses import dataclass
14     from pathlib import Path
15
16     import pypdf
17
18
19     @dataclass
20     class UnlockResult:
21         ok: bool
22         method: str | None # "plaintext" | "candidate" | "prompt"
23         password: str | None # working password ? do NOT log; None if plaintext/failed
24         output_path: Path | None # decrypted copy, if a destination was given
25
26
27     def is_encrypted(path: Path | str) -> bool:
28         return pypdf.PdfReader(str(path)).is_encrypted
29
30
31     def password_works(path: Path | str, password: str) -> bool:
32         reader = pypdf.PdfReader(str(path))
33         if not reader.is_encrypted:
34             return True
35         try:
36             return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
37         except Exception:
38             return False
39
40
41     def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
42         """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""

```

```

43     src = Path(path)
44     dest = Path(dest)
45     dest.parent.mkdir(parents=True, exist_ok=True)
46     reader = pypdf.PdfReader(str(src))
47     if reader.is_encrypted:
48         reader.decrypt(password or "")
49     writer = pypdf.PdfWriter()
50     writer.append(reader)
51     with open(dest, "wb") as fh:
52         writer.write(fh)
53     return dest
54
55
56 def unlock(
57     path: Path | str,
58     candidates: Iterable[str] = (),
59     prompt_fn: Callable[[], str | None] | None = None,
60     max_prompts: int = 3,
61     dest: Path | str | None = None,
62 ) -> UnlockResult:
63     """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.
64
65     If dest is given and unlock succeeds, a decrypted copy is written there. The
66     working password is returned in the result but must never be logged.
67     """
68     src = Path(path)
69     if not is_encrypted(src):
70         out = None
71         if dest:
72             Path(dest).parent.mkdir(parents=True, exist_ok=True) # parity with
write_decrypted
73             out = Path(shutil.copyfile(src, dest))
74             return UnlockResult(True, "plaintext", None, out)
75
76     tried: set[str] = set()
77     for pw in candidates:
78         if pw in tried:
79             continue
80         tried.add(pw)
81         if password_works(src, pw):
82             out = write_decrypted(src, pw, dest) if dest else None
83             return UnlockResult(True, "candidate", pw, out)
84
85     if prompt_fn is not None:
86         for _ in range(max_prompts):
87             entered = prompt_fn()
88             if not entered:
89                 break
90             if password_works(src, entered):
91                 out = write_decrypted(src, entered, dest) if dest else None
92                 return UnlockResult(True, "prompt", entered, out)
93
94     return UnlockResult(False, None, None, None)
95

```

18. user (2026-06-10T18:00:12.332Z)

```

1     """PDF text extraction (the deterministic, born-digital path).
2
3     Uses **pdfminer.six (MIT)** ? a permissive, commercially-safe, pure-Python text
4     extractor. We deliberately do not use PyMuPDF/`fitz`: it is AGPL-3.0,
5     incompatible with muneem's commercial-safety principle (see docs/DEPENDENCIES.md).
6
7     NFKC normalization is applied because the real corpus returns ligatures
8     (e.g. U+FB00 '?') that would otherwise break naive matching. Image extraction +

```

```

9      OCR (for codes rasterized into banners, e.g. Swiggy's CTSWIGGYUX) is the image
10     tier and lands with the OCR work (pypdfium2 + Tesseract, both permissive).
11     """
12
13     from __future__ import annotations
14
15     import hashlib
16     import unicodedata
17     from dataclasses import dataclass
18     from pathlib import Path
19
20     from pdfminer.high_level import extract_text
21
22
23     @dataclass
24     class Extracted:
25         """The result of reading a PDF's text layer."""
26
27         path: Path
28         sha256: str
29         text: str
30
31
32     def _normalize(text: str) -> str:
33         """NFKC-normalize so ligatures fold to ASCII (U+FB00 '?' -> 'ff')."""
34         return unicodedata.normalize("NFKC", text)
35
36
37     def file_sha256(path: Path | str) -> str:
38         """Stream the file and return its hex SHA-256 (used for document dedup)."""
39         digest = hashlib.sha256()
40         with open(path, "rb") as fh:
41             for chunk in iter(lambda: fh.read(65536), b''):
42                 digest.update(chunk)
43         return digest.hexdigest()
44
45
46     def extract(pdf_path: Path | str, password: str = "") -> Extracted:
47         """Extract and NFKC-normalize the full text layer of a PDF.
48
49         ``password`` is forwarded to pdfminer for encrypted statements and used in-memory
50         only ? it is
51         never stored or logged. The sha256 is over the file bytes, so it is independent of
52         the password.
53         """
54         path = Path(pdf_path)
55         text = extract_text(str(path), password=password or "")
56         return Extracted(path=path, sha256=file_sha256(path), text=_normalize(text))

```

19. user (2026-06-10T18:00:12.467Z)

```

1      from __future__ import annotations
2
3      from .base import StatementParser, safe_amount
4
5      _REGISTRY: list[type[StatementParser]] = []
6
7
8      def register_parser(parser_cls: type[StatementParser]) -> type[StatementParser]:
9          """Decorator to register a parser."""
10         _REGISTRY.append(parser_cls)
11         return parser_cls
12
13

```

```

14     def get_parser(
15         text: str, sender: str | None = None, subject: str | None = None
16     ) -> StatementParser | None:
17         """Return an instantiated parser that can handle this document, or None."""
18         for parser_cls in _REGISTRY:
19             parser = parser_cls()
20             if parser.can_handle(text, sender, subject):
21                 return parser
22         return None
23
24
25     def _autoload_parsers() -> None:
26         """Import every parser submodule so its ``@register_parser`` runs.
27
28         Registration is an import *side-effect*, so the registry is only populated for
29         modules that have actually been imported. Without this, the production path
30         (``from muneem.parsers import get_parser``, as cli.py uses) would leave
31         ``_REGISTRY`` EMPTY ? only the tests imported the submodules. Auto-discovery
32         fixes that and means new parser modules register themselves with no edit here.
33         """
34         import importlib
35         import pkgutil
36
37         for info in pkgutil.iter_modules(__path__):
38             if info.name != "base" and not info.name.startswith("_"):
39                 importlib.import_module(f"{__name__}.{info.name}")
40
41
42     _autoload_parsers()
43
44     __all__ = ["StatementParser", "register_parser", "get_parser", "safe_amount"]
45

```

20. user (2026-06-10T18:01:22.905Z)

```

407: def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
408-     return conn.execute("SELECT * FROM documents WHERE sha256 = ?", (sha256,)).fetchone()
409-
410-
411: def clear_document_rows(conn: sqlite3.Connection, document_id: int) -> None:
412-     """Remove every per-issuer transaction row tied to a document (pre-re-parse reset).
413-
414-     Used when a previously quarantined/suspect document is re-ingested after a parser fix:
415-     the parse starts from a clean slate under the SAME document id (sha256 stays the dedup
416-     key). Store-nothing parsers leave no rows, so this usually deletes nothing; legacy
417-     residue from before quarantine-parity is cleared too. Provenance upserts on re-parse.
418-     """
419-     for table in (
420-         "axis_transactions",
421-         "axis_cc_transactions",
422-         "au_transactions",
423-         "hdfc_transactions",
424-     ):
425-         conn.execute(f"DELETE FROM {table} WHERE document_id = ?", (document_id,))
426-
427-
428: def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
429-     """Internal helper for the simple INSERT pattern used by documents, offers, and bank
430-     txns."""
431-     cols = ", ".join(fields)
432-     placeholders = ", ".join("?" for _ in fields)
433-     cur = conn.execute(
434-         f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
435-         tuple(fields.values()),
436-     )

```

```

436-     rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
checker
--
442: def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
443-     """Insert into documents table. Delegates to the shared helper for DRY."""
444-     return _insert_row(conn, "documents", dict(fields))
445-
446-
447: def record_provenance(
448-     conn: sqlite3.Connection,
449-     *,
450-     document_id: int,
451-     source_type: str,
452-     parser: str | None = None,
453-     parser_version: str | None = None,
454-     mapping_layer: str | None = None,
455-     confidence: str | None = None,
456-) -> None:
457-     """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
458-
459-     First-class but *not* per transaction row: every txn table FKs ``documents(id)``, so a
single
460-     provenance record per document is reachable by join. Idempotent on ``document_id`` (a
re-parse
461-     overwrites rather than duplicating).
462-     """
463-     conn.execute(
464-         "INSERT INTO provenance "
465-         "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
466-         "VALUES (?, ?, ?, ?, ?, ?) "
467-         "ON CONFLICT(document_id) DO UPDATE SET "
468-
469-
470-
471-
472-
473-
474-
475: def record_parse(
476-     conn: sqlite3.Connection,
477-     document_id: int,
478-     *,
479-     confidence: str,
480-     parser: str,
481-     source_type: str = "pdf_statement",
482-     parser_version: str | None = None,
483-     mapping_layer: str = "deterministic",
484-) -> None:
485-     """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
486-     conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence, document_id))
487-     record_provenance(
488-         conn,
489-         document_id=document_id,
490-         source_type=source_type,
491-         parser=parser,
492-         parser_version=parser_version,
493-         mapping_layer=mapping_layer,
494-         confidence=confidence,
495-     )
496-
497-
498: def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
499-     return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518: def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
519-     return _insert_row(conn, "axis_transactions", fields)
520-
521-

```

```

522: def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
523-     return _insert_row(conn, "axis_cc_transactions", fields)
524-
525-
526-# Per-bank listings carry the parse verdict (documents.status as doc_status) so a consumer
can
527-# tell trusted rows from quarantined/legacy residue without a second query.
528: def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
529-     return conn.execute(
530-         "SELECT t.*, d.status AS doc_status FROM axis_cc_transactions t "
531-         "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
532-         (limit,),
533-     ).fetchall()
534-
535-
536: def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
537-     return conn.execute(
538-         "SELECT t.*, d.status AS doc_status FROM axis_transactions t "
539-         "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
540-         (limit,),
541-     ).fetchall()
542-
543-
544: def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
545-     return _insert_row(conn, "hdfc_transactions", fields)
546-
547-
548: def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
549-     return conn.execute(
550-         "SELECT t.*, d.status AS doc_status FROM hdfc_transactions t "
551-         "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
552-         (limit,),
553-     ).fetchall()
554-
555-
556: def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
557-     return _insert_row(conn, "au_transactions", fields)
558-
559-
560: def list_all_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
561-     """Cross-bank rows from the derived serving view, with each parse's confidence joined
in.
562-
563-     Read-only over ``v_transactions`` (migration v6) ? the flat per-issuer tables remain the
564-     source of truth (DESIGN §8.7).
565-     """
566-     return conn.execute(
567-         """
568-         SELECT v.*, p.confidence
569-         FROM v_transactions v
570-         LEFT JOIN provenance p ON p.document_id = v.document_id
571-         ORDER BY v.document_id DESC
572-         LIMIT ?
573-         """,
574-         (limit,),
575-     ).fetchall()
576-
577-
578: def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
579-     return conn.execute(
580-         "SELECT t.*, d.status AS doc_status FROM au_transactions t "
581-         "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
582-         (limit,),
583-     ).fetchall()

```

21. user (2026-06-10T18:01:23.396Z)

```
1      [build-system]
2      requires = ["setuptools>=61"]
3      build-backend = "setuptools.build_meta"
4
5      [project]
6      name = "muneem"
7      description = "A local-first personal-finance bookkeeper that ingests financial
documents from email into a consolidated ledger."
8      readme = "README.md"
9      requires-python = ">=3.11"
10     dynamic = ["version"]
11     # Private, single-user tool ? this classifier makes packaging tools refuse to upload it
to PyPI.
12     classifiers = ["Private :: Do Not Upload"]
13     dependencies = [
14         "typer>=0.12",
15         "platformdirs>=4.0",
16         "keyring>=25.0",
17         "pdfminer.six>=20240706",
18         "pypdfium2>=4.30",
19         "pytesseract>=0.3.10",
20         "pillow>=10.0",
21         "pypdf>=4.0",
22         "google-api-python-client>=2.100",
23         "google-auth>=2.30",
24         "google-auth-oauthlib>=1.2",
25         "pdfplumber>=0.11",
26         "sqlcipher3>=0.6",
27         "cryptography>=42",
28     ]
29
30     [project.scripts]
31     muneem = "muneem.cli:app"
32
33     [project.optional-dependencies]
34     outlook = ["msal>=1.0", "requests>=2.0"]
35     dev = [
36         "pytest>=8.0",
37         "pytest-cov>=5.0",
38         "ruff>=0.6",
39         "pyyaml>=6.0",
40         "mypy>=1.8",
41     ]
42
43     [tool.setuptools.dynamic]
44     version = { attr = "muneem.__version__" }
45
46     [tool.setuptools.packages.find]
47     where = ["src"]
48
49     [tool.ruff]
50     line-length = 100
51     src = ["src", "tests"]
52
53     [tool.ruff.lint]
54     select = ["E", "F", "I", "UP", "B"]
55
56     [tool.ruff.format]
57     quote-style = "double"
58     indent-style = "space"
59     skip-magic-trailing-comma = false
60     line-ending = "auto"
61
```

```

62     [tool.ruff.lint.flake8-bugbear]
63     # Typer uses Option()/Argument() as parameter defaults by design.
64     extend-immutable-calls = ["typer.Option", "typer.Argument"]
65
66     [tool.mypy]
67     files = ["src/muneem"]
68     python_version = "3.11"
69     ignore_missing_imports = true    # not all third-party libs ship type stubs
70     check_untyped_defs = true
71     no_implicit_optional = true
72     warn_redundant_casts = true
73     warn_unused_ignores = true
74     warn_unused_configs = true
75
76     # The email-ingestion providers (gmail/imap/outlook) are experimental scaffolding with
77     # NotImplemented methods (docs/CURRENT_STATE.md §1). Type-harden them when the `fetch`
command
78     # lands (ROADMAP B.4); until then don't gate CI on their half-built signatures.
79     [[tool.mypy.overrides]]
80     module = "muneem.ingestion.*"
81     ignore_errors = true
82
83     [tool.pytest.ini_options]
84     # Default runs are offline and deterministic: the ocr/llm/local_statements tiers
85     # (see docs/TESTING.md) are opt-in and excluded here.
86     addopts = "-ra -m 'not ocr and not llm and not local_statements'"
87     testpaths = ["tests"]
88     markers = [
89         "ocr: requires the Tesseract binary (image-OCR tier)",
90         "llm: makes live LLM calls (never in default CI)",
91         "local_statements: reads gitignored real statements (local only)",
92     ]
93

```

22. assistant (2026-06-10T18:04:35.034Z)

Now I have the full picture. Let me check the test environment baseline before writing code.

23. user (2026-06-10T18:04:38.964Z)

C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-a0a70ddb1d7197b83\src\muneem__init__.py
0.11.9 6.12.2

24. user (2026-06-10T18:04:42.824Z)

```

..                                                                    [100%]
2 passed in 0.48s

```

25. assistant (2026-06-10T18:04:51.164Z)

Worktree code is what's under test. Now I'll extend synthpdf with multi-page support and ruled-line drawing, then prototype the extract_tables path before writing tests.

26. assistant (2026-06-10T18:05:20.262Z)

""Dependency-free, born-digital PDF generator for regression fixtures (stdlib only).

Layer 1 of the regression-CI plan (ROADMAP § "Planned ? Regression CI") needs **full-pipeline** fixtures ? real PDFs that go through pdfminer/pdfplumber, not synthetic word-lists that bypass them.

Rather than add a dependency (reportlab etc.), we emit a minimal valid PDF with **positioned Helvetica text** (`\Tm/\Tj`), so it stays commercial-safe and CI-portable.

Coordinates: callers think in pdfplumber's top-down `top` (distance from the page top); we


```

convert
to PDF's bottom-up baseline. Absolute `top` need not be exact ? parsers key on x-positions,
same-row
grouping, and row order ? but words on one row share a baseline (one `top`) and rows are
separated.
Amount columns are **right-aligned** via real Helvetica metrics so their right edges (`x1`) land
in
the bands the HDFC clusterer / engine expect.

```

```

Beyond text, a page can carry stroked **ruling lines** (`m`/`l`/`S` path operators). pdfminer
turns
a stroked two-point path into an ``LTLine``, which is exactly what pdfplumber's default
`extract_tables()` ` "lines" strategy keys on ? so a :func:`grid` makes a hand-rolled table
visible
to the cell-matrix extraction path (the Axis credit-card layout). Multi-page statements are
built
with :func:`build_pdf_pages`; :func:`build_pdf` stays the single-page convenience.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass, field
from pathlib import Path
from typing import Sequence

```

Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-align (digits / comma / period / minus / space); a reasonable default covers labels (left-align their width is irrelevant to placement).

```

_W = {
    " ": 278,
    "!": 278,
    "'": 355,
    "#": 556,
    "$": 556,
    "%": 889,
    "&": 667,
    "' ': 191,
    "(": 333,
    ")": 333,
    "*": 389,
    "+": 584,
    ", ": 278,
    "- ": 333,
    ". ": 278,
    "/": 278,
    "0": 556,
    "1": 556,
    "2": 556,
    "3": 556,
    "4": 556,
    "5": 556,
    "6": 556,
    "7": 556,
    "8": 556,
    "9": 556,
    ": ": 278,
    "; ": 278,
    "< ": 584,
    "=": 584,
    "> ": 584,
    "?": 556,
    "@": 1015,
}

```

```
_UPPER = 667 # rough average for A-Z
_LOWER = 528 # rough average for a-z
```

```
def helvetica_width(text: str, size: float) -> float:
    """Advance width of ``text`` in Helvetica at ``size`` points."""
    total = 0
    for ch in text:
        if ch in _W:
            total += _W[ch]
        elif ch.isupper():
            total += _UPPER
        else:
            total += _LOWER
    return total / 1000.0 * size
```

```
@dataclass
class Placement:
    """One text run: its LEFT edge ``x``, top-down ``top``, and the string."""

    x: float
    top: float
    text: str
    size: float = 9.0
```

```
def at(x: float, top: float, text: str, size: float = 9.0) -> Placement:
    """Left-aligned run starting at ``x``."""
    return Placement(x=x, top=top, text=text, size=size)
```

```
def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
    """Right-aligned run whose right edge lands at ``x1`` (for amount columns)."""
    return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)
```

```
@dataclass
class Line:
    """One stroked straight segment, in the same top-down coordinates as :class:`Placement`.

    pdfminer reports a stroked two-point path as an ``LTLine``, which feeds pdfplumber's
    ``extract_tables()`` "lines" strategy ? draw a full :func:`grid` to make a table ruled.
    """

    x0: float
    top0: float
    x1: float
    top1: float
    width: float = 0.75
```

```
def grid(x_edges: Sequence[float], top_edges: Sequence[float]) -> list[Line]:
    """The full ruling of a table: a vertical line per column edge, horizontal per row edge.

    ``extract_tables()``'s default strategy intersects these to find cells, so every boundary
    must be drawn (a borderless or semi-ruled table is exactly what it fails on).
    """
    top_lo, top_hi = min(top_edges), max(top_edges)
    x_lo, x_hi = min(x_edges), max(x_edges)
    lines = [Line(x, top_lo, x, top_hi) for x in x_edges]
    lines += [Line(x_lo, t, x_hi, t) for t in top_edges]
    return lines
```

```

@dataclass
class Page:
    """One page's content: positioned text plus any ruling lines."""

    placements: list[Placement]
    lines: list[Line] = field(default_factory=list)

def _esc(text: str) -> str:
    return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")

def _content_stream(page: Page, height: float) -> bytes:
    """Render one page's lines + text runs as PDF content-stream operators."""
    ops: list[str] = []
    for ln in page.lines:
        # q/Q isolate the line width; top-down -> bottom-up like the text path below.
        ops.append(
            f"q {ln.width:.2f} w {ln.x0:.2f} {height - ln.top0:.2f} m "
            f"{ln.x1:.2f} {height - ln.top1:.2f} l S Q"
        )
    for p in page.placements:
        y = height - p.top # top-down -> PDF bottom-up baseline
        ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm ({_esc(p.text)}) Tj")
    ET")
    return ("\n".join(ops)).encode("latin-1")

def build_pdf_pages(
    pages: Sequence[Page], path: Path | str, *, width: float = 595.0, height: float = 842.0
) -> Path:
    """Write a multi-page PDF with the given per-page content. Returns the path.

    Hand-rolls a minimal PDF (catalog ? pages ? font ? page/content pairs) with a byte-accurate
    xref table so pdfminer/pdfplumber parse it. Object layout: 1 catalog, 2 page tree, 3 the
    shared
    Helvetica font, then each page as a (page, content) object pair.
    """
    page_obj_ids = [4 + 2 * i for i in range(len(pages))]
    kids = " ".join(f"{n} 0 R" for n in page_obj_ids)
    objects: list[bytes] = [
        b"<< /Type /Catalog /Pages 2 0 R >>",
        f"<< /Type /Pages /Kids [{kids}] /Count {len(pages)} >>".encode("latin-1"),
        b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
    ]
    for i, page in enumerate(pages):
        content = _content_stream(page, height)
        objects.append(
            (
                f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
                f"/Resources << /Font << /F1 3 0 R >> >> /Contents {page_obj_ids[i] + 1} 0 R >>"
            ).encode("latin-1")
        )
        objects.append(
            b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
            b"\nendstream"
        )

    out = bytearray(b"%PDF-1.4\n")
    offsets: list[int] = []
    for i, obj in enumerate(objects, start=1):
        offsets.append(len(out))
        out += f"{i} 0 obj\n".encode() + obj + b"\nendobj\n"

    xref_pos = len(out)

```

```

count = len(objects) + 1
out += f"xref\n0 {count}\n".encode()
out += b"0000000000 65535 f \n"
for off in offsets:
    out += f"{off:010d} 00000 n \n".encode()
out += (
    b"trailer\n<< /Size " + str(count).encode() + b" /Root 1 0 R >>\n"
    b"startxref\n" + str(xref_pos).encode() + b"\n%%EOF"
)

out_path = Path(path)
out_path.write_bytes(out)
return out_path

def build_pdf(
    placements: list[Placement], path: Path | str, *, width: float = 595.0, height: float =
842.0
) -> Path:
    """Write a single-page PDF with the given positioned text. Returns the path."""
    return build_pdf_pages([Page(placements)], path, width=width, height=height)

__all__ = [
    "Placement",
    "Line",
    "Page",
    "at",
    "right",
    "grid",
    "helvetica_width",
    "build_pdf",
    "build_pdf_pages",
]

```

27. user (2026-06-10T18:05:20.283Z)

The file

C:\Users\avidu\Projects\muneeem\claude\worktrees\agent-a0a70ddb1d7197b83\tests\synthpdf.py has been updated successfully. (file state is current in your context ? no need to Read it back)

28. user (2026-06-10T18:05:34.788Z)

lines: 15

tables: 1

```

['Date', 'Ref', 'Transaction Details', 'EMI', 'Intl', 'Cash', 'Pts', 'Merchant Category',
'Amount']
['05/01/2026', '', 'AMAZON PAY INDIA', '', '', '', '', 'ECOMMERCE', '2,499.00 Dr']
['11/01/2026', '', 'SWIGGY BANGALORE', '', '', '', '', 'RESTAURANT', '651.50 Dr']
['18/01/2026', '', 'PAYMENT RECEIVED', '', '', '', '', 'PAYMENT', '3,000.00 Cr']

```

29. assistant (2026-06-10T18:08:18.183Z)

"""Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.

Generates *real* PDFs (no committed statements, no secrets) and runs the production pipeline ?

```extract``` (pdfminer) ? ```get_parser``` ? ```parse_and_insert``` ? assert the recorded verdict.

This

covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the synthetic word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP § "Planned ? Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud CI.

Corpus breadth (deep-review D1): per-issuer layouts (HDFC, AU, Axis savings), the ruled Axis credit-card table through ```extract_tables()```, a 2-page statement with a repeated header, a

bookend-violating statement that must quarantine, and a password-protected statement driven through the real CLI hybrid-unlock path.

```
import pypdf
from typer.testing import CliRunner

from muneem import db as dbmod
from muneem import statement_unlock
from muneem.cli import app
from muneem.extract import extract
from muneem.parsers import get_parser
from muneem.unlock import is_encrypted
from synthpdf import Page, Placement, at, build_pdf, build_pdf_pages, grid, right

runner = CliRunner()
```

### --- HDFC layout (bespoke coordinate clustering; SUMMARY bookend) -----

```
def _hdfc_letterhead() -> list[Placement]:
 return [
 at(50, 60, "HDFC BANK"),
 at(50, 72, "Savings Account Details"),
 at(50, 84, "Account Number : 50100012345678"),
]

def _hdfc_table_header(top: float = 110) -> list[Placement]:
 """Column header ? must carry "Narration" + a "Closing" token so _table_body locates the
 table; repeated on every page of a real HDFC statement."""
 return [
 at(72, top, "Date"),
 at(175, top, "Narration"),
 at(300, top, "Withdrawals"),
 at(390, top, "Deposits"),
 at(480, top, "Closing"),
 at(515, top, "Balance"),
]

def _hdfc_rows(rows: list[tuple[str, str, str, str, str]], top: float = 130) -> list[Placement]:
 """Transaction rows in HDFC geometry: leading date x0?52, narration x0?111, three
 right-aligned amount columns whose right edges land in the clusterer's bands
 (withdrawal <400, deposit <490, balance ?490)."""
 p: list[Placement] = []
 for d, narr, wd, dep, bal in rows:
 p += [
 at(52, top, d),
 at(111, top, narr),
 right(358, top, wd),
 right(443, top, dep),
 right(561, top, bal),
]
 top += 15
 return p

def _hdfc_summary(
 figures: str, counts: str, top: float = 200
) -> list[Placement]:
 return [
 at(50, top, "SUMMARY"),
]
```

```

 at(50, top + 12, "Opening Balance Debit Amount Credit Amount Closing Balance"),
 at(50, top + 24, figures),
 at(50, top + 36, "Debit Count Credit Count"),
 at(50, top + 48, counts),
]

def _hdfc_pdf(path):
 """Single-page HDFC statement: three rows that walk (opening 10,000 ? closing 10,000) plus
 the SUMMARY block for the bookend."""
 rows = [
 ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
 ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
 ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
]
 p = (
 _hdfc_letterhead()
 + _hdfc_table_header()
 + _hdfc_rows(rows)
 + _hdfc_summary("10,000.00 2,000.00 2,000.00 10,000.00", "2 1")
)
 return build_pdf(p, path)

def _hdfc_multipage_pdf(path):
 """Two-page HDFC statement: the transaction band splits across pages with the column header
 repeated (as the real statements do), SUMMARY on the last page. The walk and the SUMMARY
 bookend both span the page break, so a clean verdict proves page stitching."""
 page1_rows = [
 ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
 ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
 ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
]
 page2_rows = [
 ("12/02/2026", "UPI-BIGBASKET", "800.00", "0.00", "9,200.00"),
 ("15/02/2026", "NEFT-REFUND", "0.00", "1,300.00", "10,500.00"),
]
 page1 = _hdfc_letterhead() + _hdfc_table_header() + _hdfc_rows(page1_rows)
 page2 = (
 _hdfc_table_header()
 + _hdfc_rows(page2_rows)
 + _hdfc_summary("10,000.00 2,800.00 3,300.00 10,500.00", "3 2")
)
 return build_pdf_pages([Page(page1), Page(page2)], path)

```

### --- Shared savings layout (the engine-driven savings.py path used by Axis/AU) -----

```

def _savings_pdf(
 path,
 letterhead: list[str],
 rows: list[tuple[str, str, str, str, str]],
 *,
 opening: str | None = None,
 closing: str | None = None,
):
 """Whitespace-aligned savings table: date | narration | debit | credit | balance in
 well-separated columns ? coordinate clustering ? engine mapping ? per-row reconcile.

 ``letterhead`` carries the routing markers and the account line; ``opening``/``closing``
 print the summary lines the trust oracle anchors a passing walk to
 (savings.reconcile_oracle).
 """

```

```

p: list[Placement] = []
top = 60
for line in letterhead:
 p.append(at(50, top, line))
 top += 12
header (clustering's date+money filter drops it; here only for realism / text markers)
p += [
 at(52, 100, "Date"),
 at(110, 100, "Transaction"),
 at(310, 100, "Withdrawal"),
 at(395, 100, "Deposit"),
 at(485, 100, "Balance"),
]
top = 124
for d, narr, debit, credit, bal in rows:
 line_p = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
 if debit:
 line_p.append(right(340, top, debit))
 if credit:
 line_p.append(right(420, top, credit))
 p += line_p
 top += 15
if opening is not None:
 p.append(at(50, top + 20, f"Opening Balance : {opening}"))
if closing is not None:
 p.append(at(50, top + 32, f"Closing Balance : {closing}"))
return build_pdf(p, path)

```

```
_AU_LETTERHEAD = ["AU Small Finance Bank", "Account Number : 1234567890123"]
```

## Three rows that walk from an implied opening of 50,000.00.

```

_AU_ROWS = [
 ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
 ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
 ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
]

```

```

def _au_savings_pdf(path):
 return _savings_pdf(path, _AU_LETTERHEAD, _AU_ROWS)

```

```

def _axis_savings_pdf(path):
 """Axis savings: AXIS_PROFILE dates (dd-mm-yyyy) + the letterhead markers AxisParser routes
 on, with printed Opening/Closing Balance lines consistent with the rows ? exercising the
 bookend-anchored trust oracle on the happy path."""
 rows = [
 ("03-09-2025", "UPI-FLIPKART", "5,000.00", "", "20,000.00"),
 ("10-09-2025", "NEFT-SALARY", "", "40,000.00", "60,000.00"),
 ("18-09-2025", "ATM-WDL", "10,000.00", "", "50,000.00"),
]
 return _savings_pdf(
 path,
 ["Axis Bank", "Statement of Account", "Account Number : 912345678901"],
 rows,
 opening="25,000.00",
 closing="50,000.00",
)

```

```

def _au_dropped_first_row_pdf(path):
 """The Axis May-2025 failure class, reproduced at the PDF level: the printed rows are
 internally walk-consistent (the chain 78,800 ? 76,800 holds) but the FIRST transaction of
 the real set is missing, so the first row's implied opening (48,800) contradicts the printed

```

```

Opening Balance (50,000) of the full set. Only the bookend anchoring can catch this."""
return _savings_pdf(
 path, _AU_LETTERHEAD, _AU_ROWS[1:], opening="50,000.00", closing="76,800.00"
)

```

### --- Axis credit card (ruled table through pdfplumber extract\_tables) -----

```

def _axis_cc_pdf(path):
 """Axis credit-card statement: a fully *ruled* 9-column table (a synthpdf `grid` of stroked
 lines), because the CC parser reads `page.extract_tables()` whose default "lines" strategy
 needs real ruling. Column contract per axis._rows_from_cc_tables: date col 0 (dd/mm/yyyy),
 particulars col 2, merchant category col 7, amount col 8 with a Dr/Cr suffix."""
 x_edges = [40.0, 100.0, 140.0, 280.0, 315.0, 350.0, 385.0, 420.0, 510.0, 565.0]
 tops = [150.0, 170.0, 190.0, 210.0, 230.0]
 p = [
 at(50, 60, "Axis Bank"),
 at(50, 72, "Credit Card Statement"),
 at(50, 84, "Card Number : 4321XXXX9876"),
]
 header = ("Date", "Ref", "Transaction Details", "EMI", "Intl", "Cash", "Pts",
 "Merchant Category", "Amount")
 rows = [
 ("05/01/2026", "", "AMAZON PAY INDIA", "", "", "", "", "ECOMMERCE", "2,499.00 Dr"),
 ("11/01/2026", "", "SWIGGY BANGALORE", "", "", "", "", "RESTAURANT", "651.50 Dr"),
 ("18/01/2026", "", "PAYMENT RECEIVED", "", "", "", "", "PAYMENT", "3,000.00 Cr"),
]
 for table_row, cells in enumerate((header, *rows)):
 for col, text in enumerate(cells):
 if text:
 p.append(at(x_edges[col] + 3, tops[table_row] + 14, text, size=8))
 return build_pdf_pages([Page(p, lines=grid(x_edges, tops))], path)

```

### --- The pipeline under test -----

```

_TXN_TABLES = ("axis_transactions", "axis_cc_transactions", "au_transactions",
 "hdfc_transactions")

```

```

def _run_pipeline(pdf) -> tuple[str, int, str, int]:
 """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back the
 verdict.

 Returns `(parser name, rows reported, document status, rows actually stored across every
 per-issuer table)` ? the stored count lets quarantine tests prove nothing landed.
 """
 ex = extract(pdf)
 parser = get_parser(ex.text)
 assert parser is not None, "no parser routed this synthetic statement"
 with dbmod.connect(":memory:") as conn:
 dbmod.init_db(conn)
 doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256, status="parsed")
 n = parser.parse_and_insert(pdf, conn, doc_id)
 status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
 stored = sum(
 conn.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0] for t in _TXN_TABLES
)
 return type(parser).__name__, n, status, stored

```



```

def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
 name, n, status, stored = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
 assert name == "HDFCParser"
 assert n == 3, f"expected 3 transactions, got {n}"
 assert status == "reconciled", f"expected reconciled, got {status}"
 assert stored == 3

def test_savings_layout_full_pipeline_reconciles(tmp_path):
 name, n, status, stored = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
 assert name == "AUParser"
 assert n == 3, f"expected 3 transactions, got {n}"
 assert status == "reconciled", f"expected reconciled, got {status}"
 assert stored == 3

def test_axis_savings_layout_full_pipeline_reconciles(tmp_path):
 """Axis savings routes via the title-case letterhead evidence and reconciles through the
 shared engine path, with the printed bookends anchoring the walk."""
 name, n, status, stored = _run_pipeline(_axis_savings_pdf(tmp_path / "axis.pdf"))
 assert name == "AxisParser"
 assert n == 3, f"expected 3 transactions, got {n}"
 assert status == "reconciled", f"expected reconciled, got {status}"
 assert stored == 3

def test_axis_cc_ruled_table_persists_unverified(tmp_path):
 """A CC bill has no running balance, so no oracle can endorse it: rows persist but the
 document is explicitly 'unverified' (Confidence.UNVERIFIED), never silently trusted."""
 name, n, status, stored = _run_pipeline(_axis_cc_pdf(tmp_path / "axis-cc.pdf"))
 assert name == "AxisCreditCardParser"
 assert n == 3, f"expected 3 transactions, got {n}"
 assert status == "unverified", f"expected unverified, got {status}"
 assert stored == 3

def test_multipage_hdfc_reconciles_across_page_break(tmp_path):
 """Rows split across two pages (header repeated) must stitch into one combined set: the
 balance walk and the last-page SUMMARY bookend both span the page break."""
 name, n, status, stored = _run_pipeline(_hdfc_multipage_pdf(tmp_path / "hdfc-2p.pdf"))
 assert name == "HDFCParser"
 assert n == 5, f"expected 5 transactions, got {n}"
 assert status == "reconciled", f"expected reconciled, got {status}"
 assert stored == 5

def test_dropped_first_row_quarantines_at_pdf_level(tmp_path):
 """A statement missing its first transaction still walks internally ? only the printed
 Opening Balance exposes the gap. The pipeline must store NOTHING and record 'unreconciled'
 (the bookend anchoring in savings.reconcile_oracle)."""
 name, n, status, stored = _run_pipeline(_au_dropped_first_row_pdf(tmp_path / "au-gap.pdf"))
 assert name == "AUParser"
 assert n == 0, f"expected 0 rows reported, got {n}"
 assert status == "unreconciled", f"expected unreconciled, got {status}"
 assert stored == 0, f"quarantine must persist nothing, found {stored} row(s)"

def test_encrypted_statement_through_real_cli_hybrid_unlock(tmp_path, monkeypatch):
 """A password-protected statement driven through the REAL `muneem ingest` CLI: pypdf
 encryption on a synthetic AU statement, unlocked by the genuine hybrid path (the keychain
 candidate cache, seeded the same way a successful prompt-unlock caches a password ?
 resolve_password itself is NOT patched). Asserts exit 0 and reconciled rows in the ledger.
 """
 plain = _au_savings_pdf(tmp_path / "plain.pdf")
 enc = tmp_path / "statement.pdf"

```

```

reader = pypdf.PdfReader(str(plain))
writer = pypdf.PdfWriter()
writer.append(reader)
writer.encrypt("TEST1234") # synthetic fixture password ? not real material
with open(enc, "wb") as fh:
 writer.write(fh)
assert is_encrypted(enc), "the fixture must actually exercise the unlock path"

Hermetic: a real password_rules.yaml on the dev machine must not feed candidates.
monkeypatch.setattr("muneem.password_rules.CONFIG_DIR", tmp_path / "no-config")
monkeypatch.chdir(tmp_path)
Seed the keychain candidate cache through the legitimate seam (conftest isolates the
keychain to an in-memory store, so nothing touches the real OS keychain).
statement_unlock._remember("TEST1234")

db = tmp_path / "ledger.db"
result = runner.invoke(
 app, ["ingest", str(enc), "--doc-type", "bank_statement", "--db", str(db), "--no-
prompt"]
)
assert result.exit_code == 0, result.output
assert "[reconciled]" in result.output and "3 transaction(s)" in result.output

with dbmod.connect_encrypted(db) as conn:
 rows = dbmod.list_aud_transactions(conn, limit=10)
assert len(rows) == 3
assert all(r["doc_status"] == "reconciled" for r in rows)

```

### 30. user (2026-06-10T18:08:18.209Z)

The file C:\Users\avidu\Projects\muneem\claude\worktrees\agent-a0a70ddb1d7197b83\tests\test\_regression\_synth.py has been updated successfully. (file state is current in your context ? no need to Read it back)

### 31. user (2026-06-10T18:08:30.369Z)

```

..... [100%]
7 passed in 0.83s

```

### 32. user (2026-06-10T18:08:41.932Z)

The file C:\Users\avidu\Projects\muneem\claude\worktrees\agent-a0a70ddb1d7197b83\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

### 33. user (2026-06-10T18:08:52.087Z)

```

Exit code 1
> assert n == 0, f"expected 0 rows reported, got {n}"
E AssertionError: expected 0 rows reported, got 2
E assert 2 == 0

tests\test_regression_synth.py:315: AssertionError
===== short test summary info =====
FAILED tests/test_regression_synth.py::test_dropped_first_row_quarantines_at_pdf_level
1 failed in 0.68s

```

### 34. user (2026-06-10T18:09:01.362Z)

The file C:\Users\avidu\Projects\muneem\claude\worktrees\agent-a0a70ddb1d7197b83\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

### 35. assistant (2026-06-10T18:09:06.062Z)

Now the gate: ruff, mypy, full suite with and without coverage.

36. user (2026-06-10T18:09:09.045Z)

```
Exit code 1
E501 Line too long (101 > 100)
--> tests\synthpdf.py:182:101
|
180 |)
181 | objects.append(
182 | b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream"
|
^
183 |)
|
```

Found 3 errors (2 fixed, 1 remaining).

37. user (2026-06-10T18:09:16.484Z)

The file  
C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-a0a70ddb1d7197b83\tests\synthpdf.py has  
been updated successfully. (file state is current in your context ? no need to Read it back)

38. user (2026-06-10T18:09:37.311Z)

All checks passed!  
1 file reformatted, 80 files left unchanged  
Success: no issues found in 34 source files

39. user (2026-06-10T18:09:54.576Z)

src\muneem\parsers\savings.py	133	6	95%
src\muneem\parsers\validation.py	92	2	98%
src\muneem\password_rules.py	41	1	98%
src\muneem\recovery.py	63	0	100%
src\muneem\redaction.py	23	0	100%
src\muneem\secrets.py	20	0	100%
src\muneem\statement_unlock.py	45	6	87%
src\muneem\unlock.py	59	5	92%
-----			
TOTAL	2158	250	88%

Required test coverage of 80% reached. Total coverage: 88.42%  
514 passed, 6 deselected in 7.25s

40. user (2026-06-10T18:10:07.652Z)

```
..... [98%]
..... [100%]
514 passed, 6 deselected in 4.48s
```

41. assistant (2026-06-10T18:10:15.490Z)

All green: 514 passed, coverage 88.42%. Now ship.

42. user (2026-06-10T18:10:18.179Z)

```
M tests/synthpdf.py
M tests/test_regression_synth.py
tests/synthpdf.py | 118 ++++++-----
tests/test_regression_synth.py | 340 ++++++-----
2 files changed, 392 insertions(+), 66 deletions(-)
```

#### 43. user (2026-06-10T18:10:33.865Z)

Switched to a new branch 'test/synthetic-corpus-breadth'  
[test/synthetic-corpus-breadth b01a71d] test: broaden the synthetic regression corpus (deep-review D1)  
2 files changed, 392 insertions(+), 66 deletions(-)

#### 44. user (2026-06-10T18:10:43.876Z)

branch 'test/synthetic-corpus-breadth' set up to track 'origin/test/synthetic-corpus-breadth'.  
remote:  
remote: Create a pull request for 'test/synthetic-corpus-breadth' on GitHub by visiting:  
  
remote: <https://github.com/avidullu/muneem/pull/new/test/synthetic-corpus-breadth>  
  
remote:  
To <https://github.com/avidullu/muneem.git>  
\* [new branch] test/synthetic-corpus-breadth -> test/synthetic-corpus-breadth

#### 45. user (2026-06-10T18:11:12.024Z)

<https://github.com/avidullu/muneem/pull/70>

#### 46. assistant (2026-06-10T18:11:29.132Z)

PR: [\\*\\*https://github.com/avidullu/muneem/pull/70\\*\\*](https://github.com/avidullu/muneem/pull/70) (branch `test/synthetic-corpus-breadth`, not merged)

**\*\*Deliverables ? all 5 shipped, none blocked:\*\***

1. **\*\*Axis savings\*\*** ? born-digital fixture routed via title-case letterhead ("Axis Bank" + "Statement of Account") to `AxisParser`, through coordinate clustering + L2 engine search, reconciled with printed Opening/Closing Balance lines anchoring the walk. 3 rows persisted, `reconciled`.
2. **\*\*Axis CC\*\*** ? true PDF route through `extract\_tables()` worked: added `Line`/`grid` primitives to `tests/synthpdf.py` that stroke `m`/`l`/`S` paths (pdfminer ? `LTLine` ? pdfplumber "lines" strategy). The 9-column ruled table extracts cleanly; 3 rows persist with status `unverified`. No fallback seam needed.
3. **\*\*Multi-page HDFC\*\*** ? 3+2 rows across 2 pages, header repeated, SUMMARY on page 2; walk + bookend span the page break. 5 rows, `reconciled`. (`synthpdf` gained `build\_pdf\_pages`/`Page`; `build\_pdf` unchanged for existing callers.)
4. **\*\*Dropped-row quarantine\*\*** ? AU statement, internally walk-consistent but missing the first transaction vs the printed Opening Balance: 0 rows stored, `unreconciled`. Verified by mutation: disabling the opening-anchor in `savings.reconcile\_oracle` made it wrongly persist 2 rows ? the test exercises exactly the new bookend anchoring (mutation reverted).
5. **\*\*Encrypted-PDF CLI\*\*** ? pypdf RC4-128 encryption on a synthetic AU statement, driven through real `muneem ingest --doc-type bank\_statement --no-prompt` via `CliRunner`. `resolve\_password` not patched; candidate seeded via `statement\_unlock.\_remember` against the conf test in-memory keychain. Exit 0, `[reconciled]`, 3 rows read back from the encrypted ledger.

**\*\*Test counts:\*\*** 7 tests in `tests/test\_regression\_synth.py` (up from 2); full default suite 514 passed, 6 deselected; coverage 88.42% (gate 80). `ruff check`, `ruff format`, `mypy src` all clean. Verified the worktree src (not the editable install) was under test via the mutation check.

**\*\*Caveats:\*\*** the CLI test pins `muneem.password\_rules.CONFIG\_DIR` to an empty tmp dir + `chdir(tmp\_path)` for hermeticity (a real `password\_rules.yaml` on the dev machine would otherwise add unlock candidates ? harmless, since the seeded cache is tried first, but nondeterministic). `\_run\_pipeline`'s return shape changed (now includes stored-row count), updating the two pre-existing tests in the same file. CC encryption uses pypdf's default RC4-128 ? decryptable by pdfminer/pdfplumber with no extra backend.